

Assignment 4

Hadeer Ghoneim

Verification Diploma

Deliverables:

One PDF file having the following.

1. Testbench code
2. Top module code
3. SVA module code
4. Package code
5. Design code
6. Do File & Snippet to your verification requirement document
7. Code Coverage, Functional Coverage and assertion coverage report snippets
8. QuestaSim snippets

Q1

code_cvr_alsu - Notepad

File Edit Format View Help

Assertion Coverage:

Assertions	2	2	0	100.00%
------------	---	---	---	---------

Name	File(Line)	Failure Count	Pass Count
------	------------	---------------	------------

/question_3_tb/assert__1	question_3_tb.svh(65)	0	1
/question_3_tb/assert__0	question_3_tb.svh(60)	0	1

Branch Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
Branches	8	5	3	62.50%

=====Branch Details=====

Branch Coverage for instance /question_3_tb

Line	Item	Count	Source
------	------	-------	--------

File question_3_tb.svh

-----IF Branch-----

51		1095	Count coming in to IF
51	1	309	if (!rst && !bypass_A && !bypass_B) begin
		786	All False Count

Branch totals: 2 hits of 2 branches = 100.00%

-----IF Branch-----

160		1000	Count coming in to IF
160	1	***0***	if (!t.randomize()) begin
		1000	All False Count

Branch totals: 1 hit of 2 branches = 50.00%

-----IF Branch-----

189		1	Count coming in to IF
189	1	***0***	if (!t.randomize()) begin
191	1	1	end else begin

Branch totals: 1 hit of 2 branches = 50.00%

```

-----IF Branch-----
196                                     36   Count coming in to IF
196          1          ***0***          if (!t.randomize()) begin

                                     36   All False Count

```

Branch totals: 1 hit of 2 branches = 50.00%

Condition Coverage:

Enabled Coverage	Bins	Covered	Misses	Coverage
-----	----	----	-----	-----
Conditions	3	2	1	66.66%

=====Condition Details=====

Condition Coverage for instance /question_3_tb --

File question_3_tb.svh

-----Focused Condition View-----

Line 51 Item 1 ((rst ~| bypass_A) && ~bypass_B)

Condition totals: 2 of 3 input terms covered = 66.66%

Input Term	Covered	Reason for no coverage	Hint
-----	-----	-----	-----
rst	N	'_1' not hit	Hit '_1'
bypass_A	Y		
bypass_B	Y		

Rows:	Hits	FEC Target	Non-masking condition(s)
-----	-----	-----	-----
Row 1:	1	rst_0	(~bypass_B && ~bypass_A)
Row 2:	***0***	rst_1	~bypass_A
Row 3:	1	bypass_A_0	(~bypass_B && ~rst)
Row 4:	1	bypass_A_1	~rst
Row 5:	1	bypass_B_0	(rst ~ bypass_A)
Row 6:	1	bypass_B_1	(rst ~ bypass_A)

ASSERTION RESULTS:

Name	File(Line)	Failure Count	Pass Count
-----	-----	-----	-----
/question_3_tb/assert_1			
	question_3_tb.svh(65)	0	1
/question_3_tb/assert_0			
	question_3_tb.svh(60)	0	1

Total Coverage By Instance (filtered view): 90.80%

Coverage Report by instance with details

```
=====
=== Instance: /question_3_tb/DUT
=== Design Unit: work.ALSU
=====
```

Branch Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Branches	31	31	0	100.00%

=====Branch Details=====

Branch Coverage for instance /question_3_tb/DUT

Line	Item	Count	Source
----	----	----	-----
File ALSU.v			
-----IF Branch-----			
24		1038	Count coming in to IF
24	1	2	if(rst) begin
35	1	1036	end else begin

Branch totals: 2 hits of 2 branches = 100.00%

-----IF Branch-----			
51		1039	Count coming in to IF
51	1	3	if(rst) begin
53	1	1036	end else begin

Branch totals: 2 hits of 2 branches = 100.00%

-----IF Branch-----			
54		1036	Count coming in to IF
54	1	511	if (invalid)
56	1	525	else

Branch totals: 2 hits of 2 branches = 100.00%

```
-----IF Branch-----
63          1038    Count coming in to IF
63          1      2      if(rst) begin

66          1036    else begin

Branch totals: 2 hits of 2 branches = 100.00%

-----IF Branch-----
67          1036    Count coming in to IF
67          1      226   if (bypass_A_reg && bypass_B_reg)

69          262     else if (bypass_A_reg)

71          249     else if (bypass_B_reg)

73          134     else if (invalid)

75          165     else begin

Branch totals: 5 hits of 5 branches = 100.00%

-----CASE Branch-----
76          165    Count coming in to CASE
77          1      50      3'h0: begin

87          44      3'h1: begin

97          15      3'h2: out <= (FULL_ADDER == "ON") ? (A_reg + B_reg + cin_reg) : (A_reg + B_reg); //edited design error

98          15      3'h3: out <= A_reg * B_reg;

99          27      3'h4: begin

105         14      3'h5: begin

Branch totals: 6 hits of 6 branches = 100.00%

-----IF Branch-----
78          50    Count coming in to IF
78          1      7      if (red_op_A_reg && red_op_B_reg)

80          8      else if (red_op_A_reg)
```

84	1	26	else
----	---	----	------

Branch totals: 4 hits of 4 branches = 100.00%

```
-----IF Branch-----
88                                44    Count coming in to IF
88                                10      if (red_op_A_reg && red_op_B_reg)
                                13      else if (red_op_A_reg)
90                                9       else if (red_op_B_reg)
92                                12      else
94
```

Branch totals: 4 hits of 4 branches = 100.00%

```
-----IF Branch-----
100                               27    Count coming in to IF
100                               13      if (direction_reg)
102                               14      else
```

Branch totals: 2 hits of 2 branches = 100.00%

```
-----IF Branch-----
106                               14    Count coming in to IF
106                               10      if (direction_reg)
108                               4       else
```

Branch totals: 2 hits of 2 branches = 100.00%

Condition Coverage:				
Enabled Coverage	Bins	Covered	Misses	Coverage
-----	----	----	-----	-----
Conditions	6	6	0	100.00%

=====Condition Details=====

Condition Coverage for instance /question_3_tb/DUT --

code_cvr_alu - Notepad

File Edit Format View Help

=====Condition Details=====

Condition Coverage for instance /question_3_tb/DUT --

File ALSU.v

-----Focused Condition View-----

Line 67 Item 1 (bypass_A_reg && bypass_B_reg)

Condition totals: 2 of 2 input terms covered = 100.00%

Input Term	Covered	Reason for no coverage	Hint
bypass_A_reg	Y		
bypass_B_reg	Y		

Rows:	Hits	FEC Target	Non-masking condition(s)
Row 1:	1	bypass_A_reg_0	-
Row 2:	1	bypass_A_reg_1	bypass_B_reg
Row 3:	1	bypass_B_reg_0	bypass_A_reg
Row 4:	1	bypass_B_reg_1	bypass_A_reg

-----Focused Condition View-----

Line 78 Item 1 (red_op_A_reg && red_op_B_reg)

Condition totals: 2 of 2 input terms covered = 100.00%

Input Term	Covered	Reason for no coverage	Hint
red_op_A_reg	Y		
red_op_B_reg	Y		

Rows:	Hits	FEC Target	Non-masking condition(s)
Row 1:	1	red_op_A_reg_0	-
Row 2:	1	red_op_A_reg_1	red_op_B_reg
Row 3:	1	red_op_B_reg_0	red_op_A_reg
Row 4:	1	red_op_B_reg_1	red_op_A_reg

-----Focused Condition View-----

Line 88 Item 1 (red_op_A_reg && red_op_B_reg)

Condition totals: 2 of 2 input terms covered = 100.00%

Input Term	Covered	Reason for no coverage	Hint
------------	---------	------------------------	------

code_cvr_alu - Notepad

File Edit Format View Help

Condition totals: 2 of 2 input terms covered = 100.00%

Input Term	Covered	Reason for no coverage	Hint
red_op_A_reg	Y		
red_op_B_reg	Y		

Rows:	Hits	FEC Target	Non-masking condition(s)
Row 1:	1	red_op_A_reg_0	-
Row 2:	1	red_op_A_reg_1	red_op_B_reg
Row 3:	1	red_op_B_reg_0	red_op_A_reg
Row 4:	1	red_op_B_reg_1	red_op_A_reg

Expression Coverage:

Enabled Coverage	Bins	Covered	Misses	Coverage
Expressions	8	5	3	62.50%

====Expression Details=====

Expression Coverage for instance /question_3_tb/DUT --

File ALSU.v

-----Focused Expression View-----

Line 18 Item 1 ((red_op_A_reg | red_op_B_reg) & (opcode_reg[1] | opcode_reg[2]))

Expression totals: 4 of 4 input terms covered = 100.00%

Input Term	Covered	Reason for no coverage	Hint
red_op_A_reg	Y		
red_op_B_reg	Y		
opcode_reg[1]	Y		
opcode_reg[2]	Y		

Rows:	Hits	FEC Target	Non-masking condition(s)
Row 1:	1	red_op_A_reg_0	((opcode_reg[1] opcode_reg[2]) && ~red_op_B_reg)
Row 2:	1	red_op_A_reg_1	((opcode_reg[1] opcode_reg[2]) && ~red_op_B_reg)
Row 3:	1	red_op_B_reg_0	((opcode_reg[1] opcode_reg[2]) && ~red_op_A_reg)
Row 4:	1	red_op_B_reg_1	((opcode_reg[1] opcode_reg[2]) && ~red_op_A_reg)
Row 5:	1	opcode_reg[1]_0	((red_op_A_reg red_op_B_reg) && ~opcode_reg[2])

code_cvr_alu - Notepad

File Edit Format View Help

ROW	5:	1	opcode_reg[1]_0	((red_op_A_reg red_op_B_reg) && ~opcode_reg[2])
ROW	6:	1	opcode_reg[1]_1	((red_op_A_reg red_op_B_reg) && ~opcode_reg[2])
ROW	7:	1	opcode_reg[2]_0	((red_op_A_reg red_op_B_reg) && ~opcode_reg[1])
ROW	8:	1	opcode_reg[2]_1	((red_op_A_reg red_op_B_reg) && ~opcode_reg[1])

-----Focused Expression View-----

Line 19 Item 1 (opcode_reg[1] & opcode_reg[2])

Expression totals: 0 of 2 input terms covered = 0.00%

Input Term	Covered	Reason for no coverage	Hint
opcode_reg[1]	N	'_1' not hit	Hit '_1'
opcode_reg[2]	N	'_1' not hit	Hit '_1'

Rows:	Hits	FEC Target	Non-masking condition(s)
ROW 1:	1	opcode_reg[1]_0	opcode_reg[2]
ROW 2:	***0***	opcode_reg[1]_1	opcode_reg[2]
ROW 3:	1	opcode_reg[2]_0	opcode_reg[1]
ROW 4:	***0***	opcode_reg[2]_1	opcode_reg[1]

-----Focused Expression View-----

Line 20 Item 1 (invalid_red_op | invalid_opcode)

Expression totals: 1 of 2 input terms covered = 50.00%

Input Term	Covered	Reason for no coverage	Hint
invalid_red_op	Y		
invalid_opcode	N	'_1' not hit	Hit '_1'

Rows:	Hits	FEC Target	Non-masking condition(s)
ROW 1:	1	invalid_red_op_0	~invalid_opcode
ROW 2:	1	invalid_red_op_1	~invalid_opcode
ROW 3:	1	invalid_opcode_0	~invalid_red_op
ROW 4:	***0***	invalid_opcode_1	~invalid_red_op

Statement Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
Statements	48	48	0	100.00%

=====Statement Details=====

Statement Coverage for instance /question_3_tb/DUT --

Line	Item	Count	Source
-----	-----	-----	-----
File ALSU.v			
1			module ALSU(A, B, cin, serial_in, red_op_A, red_op_B, opcode, bypass_A, bypass_B, clk, rst, direction, leds, out);
2			parameter INPUT_PRIORITY = "A";
3			parameter FULL_ADDER = "ON";
4			input clk, cin, rst, red_op_A, red_op_B, bypass_A, bypass_B, direction, serial_in;
5			input [2:0] opcode;
6			input signed [2:0] A, B;
7			output reg [15:0] leds;
8			output reg signed [5:0] out;
9			
10			reg red_op_A_reg, red_op_B_reg, bypass_A_reg, bypass_B_reg, direction_reg, serial_in_reg;
11			reg signed [1:0] cin_reg;
12			reg [2:0] opcode_reg;
13			reg signed [2:0] A_reg, B_reg;
14			
15			wire invalid_red_op, invalid_opcode, invalid;
16			
17			//Invalid handling
18	1	973	assign invalid_red_op = (red_op_A_reg red_op_B_reg) & (opcode_reg[1] opcode_reg[2]);

18	1	973	assign invalid_red_op = (red_op_A_reg red_op_B_reg) & (opcode_reg[1] opcode_reg[2]);
19	1	840	assign invalid_opcode = opcode_reg[1] & opcode_reg[2];
20	1	526	assign invalid = invalid_red_op invalid_opcode;
21			
22			//Registering input signals
23	1	1038	always @(posedge clk or posedge rst) begin
24			if(rst) begin
25	1	2	cin_reg <= 0;
26	1	2	red_op_B_reg <= 0;
27	1	2	red_op_A_reg <= 0;
28	1	2	bypass_B_reg <= 0;
29	1	2	bypass_A_reg <= 0;
30	1	2	direction_reg <= 0;
31	1	2	serial_in_reg <= 0;
32	1	2	opcode_reg <= 0;
33	1	2	A_reg <= 0;
34	1	2	B_reg <= 0;
35			end else begin
36	1	1036	cin_reg <= cin;
37	1	1036	red_op_B_reg <= red_op_B;
38	1	1036	red_op_A_reg <= red_op_A;
39	1	1036	bypass_B_reg <= bypass_B;

code_cvr_alu - Notepad

File Edit Format View Help

39	1	1036	bypass_B_reg <= bypass_B;
40	1	1036	bypass_A_reg <= bypass_A;
41	1	1036	direction_reg <= direction;
42	1	1036	serial_in_reg <= serial_in;
43	1	1036	opcode_reg <= opcode;
44	1	1036	A_reg <= A;
45	1	1036	B_reg <= B;
46			end
47			end
48			
49			//leds output blinking
50	1	1039	always @(posedge clk or posedge rst) begin
51			if(rst) begin
52	1	3	leds <= 0;
53			end else begin
54			if (invalid)
55	1	511	leds <= ~leds;
56			else
57	1	525	leds <= 0;
58			end
59			end
60			

```

60
61                                //ALU output processing
62                1                1038    always @(posedge clk or posedge rst) begin
63                                if(rst) begin
64                1                2        out <= 0;
65                                end
66                                else begin
67                                if (bypass_A_reg && bypass_B_reg)
68                1                226        out <= (INPUT_PRIORITY == "A")? A_reg: B_reg;
69                                else if (bypass_A_reg)
70                1                262        out <= A_reg;
71                                else if (bypass_B_reg)
72                1                249        out <= B_reg;
73                                else if (invalid)
74                1                134        out <= 0;
75                                else begin
76                                case (opcode_reg) //edited design error
77                                3'h0: begin
78                                if (red_op_A_reg && red_op_B_reg)
79                1                7        out <= (INPUT_PRIORITY == "A")? |A_reg: |B_reg;
80                                else if (red_op_A_reg)
81                1                8        out <= |A_reg;

```

code_cvr_alu - Notepad				
File	Edit	Format	View	Help
81		1	8	out <= A_reg;
82				else if (red_op_B_reg)
83		1	9	out <= B_reg;
84				else
85		1	26	out <= A_reg B_reg;
86				end
87				3'h1: begin
88				if (red_op_A_reg && red_op_B_reg)
89		1	10	out <= (INPUT_PRIORITY == "A")? ^A_reg: ^B_reg;
90				else if (red_op_A_reg)
91		1	13	out <= ^A_reg;
92				else if (red_op_B_reg)
93		1	9	out <= ^B_reg;
94				else
95		1	12	out <= A_reg ^ B_reg;
96				end
97		1	15	3'h2: out <= (FULL_ADDER == "ON") ? (A_reg + B_reg + cin_reg) : (A_reg + B_reg); //edited design error
98		1	15	3'h3: out <= A_reg * B_reg;
99				3'h4: begin
100				if (direction_reg)
101		1	13	out <= {out[4:0], serial_in_reg};
102				else

code_cvr_alu - Notepad				
File	Edit	Format	View	Help
102				else
103		1	14	out <= {serial_in_reg, out[5:1]};
104				end
105				3'h5: begin
106				if (direction_reg)
107		1	10	out <= {out[4:0], out[5]};
108				else
109		1	4	out <= {out[0], out[5:1]};

Toggle Coverage:				
Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	----	-----
Toggles	114	114	0	100.00%

code_cvr_alu - Notepad

File Edit Format View Help

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	-----	-----	-----
Toggles	114	114	0	100.00%

=====Toggle Details=====

Toggle Coverage for instance /question_3_tb/DUT --

Node	1H->0L	0L->1H	"Coverage"
-----	-----	-----	-----
A[0-2]	1	1	100.00
A_reg[2-0]	1	1	100.00
B[0-2]	1	1	100.00
B_reg[2-0]	1	1	100.00
bypass_A	1	1	100.00
bypass_A_reg	1	1	100.00
bypass_B	1	1	100.00
bypass_B_reg	1	1	100.00
cin	1	1	100.00
cin_reg[0]	1	1	100.00
clk	1	1	100.00
direction	1	1	100.00
direction_reg	1	1	100.00
invalid	1	1	100.00
invalid_red_op	1	1	100.00
leds[15-0]	1	1	100.00
opcode[0-2]	1	1	100.00
opcode_reg[2-0]	1	1	100.00
out[5-0]	1	1	100.00
red_op_A	1	1	100.00
red_op_A_reg	1	1	100.00
red_op_B	1	1	100.00
red_op_B_reg	1	1	100.00
serial_in	1	1	100.00
serial_in_reg	1	1	100.00

Total Node Count = 57
Toggled Node Count = 57
Untoggled Node Count = 0

Toggle Coverage = 100.00% (114 of 114 bins)

=====

```

1 //new
2 module ALSU(A, B, cin, serial_in, red_op_A, red_op_B, opcode, bypass_A, bypass_B, clk, rst, direction, leds, out);
3     parameter INPUT_PRIORITY = "A";
4     parameter FULL_ADDER = "ON";
5     input clk, cin, rst, red_op_A, red_op_B, bypass_A, bypass_B, direction, serial_in;
6     input [2:0] opcode;
7     input signed [2:0] A, B;
8     output reg [15:0] leds;
9     output reg signed [5:0] out;
10
11     reg red_op_A_reg, red_op_B_reg, bypass_A_reg, bypass_B_reg, direction_reg, serial_in_reg;
12     reg signed [1:0] cin_reg;
13     reg [2:0] opcode_reg;
14     reg signed [2:0] A_reg, B_reg;
15
16     wire invalid_red_op, invalid_opcode, invalid;
17
18     // Invalid handling
19     assign invalid_red_op = (red_op_A_reg | red_op_B_reg) & (opcode_reg[1] | opcode_reg[2]);
20     assign invalid_opcode = opcode_reg[1] & opcode_reg[2];
21     assign invalid = invalid_red_op | invalid_opcode;
22
23     // Registering input signals
24     always @(posedge clk or posedge rst) begin
25         if(rst) begin
26             cin_reg <= 0;
27             red_op_B_reg <= 0;
28             red_op_A_reg <= 0;
29             bypass_B_reg <= 0;
30             bypass_A_reg <= 0;
31             direction_reg <= 0;
32             serial_in_reg <= 0;
33             opcode_reg <= 0;
34             A_reg <= 0;

```

```

35     B_reg <= 0;
36 end else begin
37     cin_reg <= cin;
38     red_op_B_reg <= red_op_B;
39     red_op_A_reg <= red_op_A;
40     bypass_B_reg <= bypass_B;
41     bypass_A_reg <= bypass_A;
42     direction_reg <= direction;
43     serial_in_reg <= serial_in;
44     opcode_reg <= opcode;
45     A_reg <= A;
46     B_reg <= B;
47 end
48 end
49
50 // leds output blinking
51 always @(posedge clk or posedge rst) begin
52     if(rst) begin
53         leds <= 0;
54     end else begin
55         if (invalid)
56             leds <= ~leds;
57         else
58             leds <= 0;
59     end
60 end
61
62 // ALSU output processing
63 always @(posedge clk or posedge rst) begin
64     if(rst) begin
65         out <= 0;
66     end

```

```

67     else begin
68         if (bypass_A_reg && bypass_B_reg)
69             out <= (INPUT_PRIORITY == "A")? A_reg : B_reg;
70         else if (bypass_A_reg)
71             out <= A_reg;
72         else if (bypass_B_reg)
73             out <= B_reg;
74         else if (invalid)
75             out <= 0;
76         else begin
77             case (opcode_reg)
78                 3'h0: begin
79                     if (red_op_A_reg && red_op_B_reg)
80                         out <= (INPUT_PRIORITY == "A")? {5'b0, |A_reg} : {5'b0, |B_reg};
81                     else if (red_op_A_reg)
82                         out <= {5'b0, |A_reg};
83                     else if (red_op_B_reg)
84                         out <= {5'b0, |B_reg};
85                     else
86                         out <= A_reg | B_reg;
87                 end
88                 3'h1: begin
89                     if (red_op_A_reg && red_op_B_reg)
90                         out <= (INPUT_PRIORITY == "A")? {5'b0, ^A_reg} : {5'b0, ^B_reg};
91                     else if (red_op_A_reg)
92                         out <= {5'b0, ^A_reg};
93                     else if (red_op_B_reg)
94                         out <= {5'b0, ^B_reg};
95                     else
96                         out <= A_reg ^ B_reg;
97                 end
98                 3'h2: out <= (FULL_ADDER == "ON") ? (A_reg + B_reg + cin_reg) : (A_reg + B_reg);
99                 3'h3: out <= A_reg * B_reg;

```

```

100         3'h4: begin
101             if (direction_reg)
102                 out <= {out[4:0], serial_in_reg};
103             else
104                 out <= {serial_in_reg, out[5:1]};
105         end
106         3'h5: begin
107             if (direction_reg)
108                 out <= {out[4:0], out[5]};
109             else
110                 out <= {out[0], out[5:1]};
111         end
112         3'h6, 3'h7: out <= 0; // Handle invalid opcodes
113         default: out <= 0;
114     endcase
115 end
116 end
117 end
118 endmodule

```

```

1  vlib work
2  vlog ALSU.v package_q_3.svh question_3_tb.svh +cover -covercells
3  vsim -voptargs=+acc work.question_3_tb -cover
4  add wave *
5  coverage save question_3_tb.ucdb -onexit
6  run -all

```

```

=====
=== Instance: /alsu_pkg
=== Design Unit: work.alsu_pkg
=====

```

Covergroup Coverage:

Covergroups	1	na	na	96.29%
Coverpoints/Crosses	17	na	na	na
Covergroup Bins	149	128	21	85.90%

Covergroup	Metric	Goal	Bins	Status
TYPE /alsu_pkg/alsu_txn/cvr_gp	96.29%	100	-	Uncovered
covered/total bins:	128	149	-	
missing/total bins:	21	149	-	
% Hit:	85.90%	100	-	
Coverpoint A_cp	100.00%	100	-	Covered
covered/total bins:	3	3	-	
missing/total bins:	0	3	-	
% Hit:	100.00%	100	-	
bin A_data_0	35	1	-	Covered
bin A_data_max	27	1	-	Covered
bin A_data_min	22	1	-	Covered
default bin A_data_default	163		-	Occurred
Coverpoint B_cp	100.00%	100	-	Covered
covered/total bins:	3	3	-	
missing/total bins:	0	3	-	
% Hit:	100.00%	100	-	
bin B_data_0	24	1	-	Covered
bin B_data_max	27	1	-	Covered
bin B_data_min	39	1	-	Covered
default bin B_data_default	157		-	Occurred
Coverpoint opcode_cp	100.00%	100	-	Covered
covered/total bins:	6	6	-	
missing/total bins:	0	6	-	
% Hit:	100.00%	100	-	
illegal_bin bins_invalid	1		-	Occurred
bin bins_shift[SHIFT_OP]	48	1	-	Covered
bin bins_shift[ROT_OP]	50	1	-	Covered
bin bins_arith[ADD_OP]	39	1	-	Covered
bin bins_arith[MUL_OP]	36	1	-	Covered
bin bins_bitwise[OR_OP]	35	1	-	Covered

code_cvr_alsu - Notepad

File Edit Format View Help

bin bins_bitwise[OR_OP]	35	1	-	Covered
bin bins_bitwise[XOR_OP]	37	1	-	Covered
Coverpoint cin_cp	100.00%	100	-	Covered
covered/total bins:	2	2	-	
missing/total bins:	0	2	-	
% Hit:	100.00%	100	-	
bin cin_0	129	1	-	Covered
bin cin_1	117	1	-	Covered
Coverpoint direction_cp	100.00%	100	-	Covered
covered/total bins:	2	2	-	
missing/total bins:	0	2	-	
% Hit:	100.00%	100	-	
bin dir_0	126	1	-	Covered
bin dir_1	120	1	-	Covered
Coverpoint serial_in_cp	100.00%	100	-	Covered
covered/total bins:	2	2	-	
missing/total bins:	0	2	-	
% Hit:	100.00%	100	-	
bin serial_0	133	1	-	Covered
bin serial_1	113	1	-	Covered
Coverpoint red_op_A_cp	100.00%	100	-	Covered
covered/total bins:	2	2	-	
missing/total bins:	0	2	-	
% Hit:	100.00%	100	-	
bin red_A_0	117	1	-	Covered
bin red_A_1	129	1	-	Covered
Coverpoint red_op_B_cp	100.00%	100	-	Covered
covered/total bins:	2	2	-	
missing/total bins:	0	2	-	
% Hit:	100.00%	100	-	
bin red_B_0	127	1	-	Covered
bin red_B_1	119	1	-	Covered
Coverpoint A_walking_cp	100.00%	100	-	Covered
covered/total bins:	2	2	-	
missing/total bins:	0	2	-	
% Hit:	100.00%	100	-	
bin walking_one[1]	29	1	-	Covered
bin walking_one[2]	31	1	-	Covered
Coverpoint B_walking_cp	100.00%	100	-	Covered
covered/total bins:	2	2	-	
missing/total bins:	0	2	-	
% Hit:	100.00%	100	-	
bin walking_one[1]	32	1	-	Covered

code_cvr_alsu - Notepad

File Edit Format View Help

bin walking_one[1]	32	1	-	Covered
bin walking_one[2]	36	1	-	Covered
Cross cross_arith_vals	51.35%	100	-	Uncovered
covered/total bins:	19	37	-	
missing/total bins:	18	37	-	
% Hit:	51.35%	100	-	
Auto, Default and User Defined Bins:				
bin add_mul_permutations	8	1	-	Covered
bin <A_data_min,B_data_min,bins_bitwise[XOR_OP]>	1	1	-	Covered
bin <A_data_min,B_data_min,bins_shift[ROT_OP]>	2	1	-	Covered
bin <A_data_min,B_data_min,bins_bitwise[OR_OP]>	1	1	-	Covered
bin <A_data_min,B_data_0,bins_shift[ROT_OP]>	1	1	-	Covered
bin <A_data_min,B_data_max,bins_bitwise[OR_OP]>	1	1	-	Covered
bin <A_data_min,B_data_0,bins_bitwise[OR_OP]>	1	1	-	Covered
bin <A_data_max,B_data_min,bins_shift[ROT_OP]>	2	1	-	Covered
bin <A_data_0,B_data_min,bins_shift[ROT_OP]>	2	1	-	Covered
bin <A_data_max,B_data_min,bins_bitwise[OR_OP]>	2	1	-	Covered
bin <A_data_0,B_data_min,bins_bitwise[OR_OP]>	1	1	-	Covered
bin <A_data_0,B_data_min,bins_shift[SHIFT_OP]>	1	1	-	Covered
bin <A_data_0,B_data_max,bins_bitwise[XOR_OP]>	1	1	-	Covered
bin <A_data_max,B_data_max,bins_shift[ROT_OP]>	1	1	-	Covered
bin <A_data_0,B_data_max,bins_shift[ROT_OP]>	2	1	-	Covered
bin <A_data_max,B_data_0,bins_shift[ROT_OP]>	1	1	-	Covered
bin <A_data_0,B_data_0,bins_bitwise[OR_OP]>	1	1	-	Covered
bin <A_data_0,B_data_max,bins_shift[SHIFT_OP]>	2	1	-	Covered
bin <A_data_max,B_data_0,bins_shift[SHIFT_OP]>				

bin <A_data_max,B_data_0,bins_shift[SHIFT_OP]>	1	1	-	Covered
bin <A_data_max,*,bins_bitwise[XOR_OP]>	0	1	3	ZERO
bin <A_data_min,*,bins_shift[SHIFT_OP]>	0	1	3	ZERO
bin <*,B_data_0,bins_bitwise[XOR_OP]>	0	1	3	ZERO
bin <A_data_0,B_data_min,bins_bitwise[XOR_OP]>	0	1	1	ZERO
bin <A_data_min,B_data_max,bins_bitwise[XOR_OP]>	0	1	1	ZERO
bin <A_data_min,B_data_max,bins_shift[ROT_OP]>	0	1	1	ZERO
bin <A_data_0,B_data_0,bins_shift[ROT_OP]>	0	1	1	ZERO
bin <A_data_0,B_data_max,bins_bitwise[OR_OP]>	0	1	1	ZERO
bin <A_data_max,B_data_max,bins_bitwise[OR_OP]>	0	1	1	ZERO
bin <A_data_max,B_data_0,bins_bitwise[OR_OP]>	0	1	1	ZERO
bin <A_data_max,B_data_min,bins_shift[SHIFT_OP]>	0	1	1	ZERO
bin <A_data_max,B_data_max,bins_shift[SHIFT_OP]>	0	1	1	ZERO
bin <A_data_0,B_data_0,bins_shift[SHIFT_OP]>	0	1	1	ZERO
Cross cross_add_cin	100.00%	100	-	Covered
covered/total bins:	11	11	-	
missing/total bins:	0	11	-	
% Hit:	100.00%	100	-	
Auto, Default and User Defined Bins:				
bin add_cin_0_1	39	1	-	Covered
bin <bins_bitwise[XOR_OP],cin_1>	17	1	-	Covered
bin <bins_bitwise[OR_OP],cin_1>	19	1	-	Covered
bin <bins_bitwise[XOR_OP],cin_0>	20	1	-	Covered
bin <bins_bitwise[OR_OP],cin_0>	16	1	-	Covered
bin <bins_arith[MUL_OP],cin_1>	18	1	-	Covered
bin <bins_arith[MUL_OP],cin_0>	18	1	-	Covered
bin <bins_shift[ROT_OP],cin_1>	23	1	-	Covered
bin <bins_shift[SHIFT_OP],cin_1>	19	1	-	Covered
bin <bins_shift[ROT_OP],cin_0>	27	1	-	Covered
bin <bins_shift[SHIFT_OP],cin_0>	29	1	-	Covered

code_cvr_alsu - Notepad

File Edit Format View Help

bin <bins_shift[SHIFT_OP],cin_0>	29	1	-	Covered
Cross cross_shift_rot_direction	100.00%	100	-	Covered
covered/total bins:	9	9	-	
missing/total bins:	0	9	-	
% Hit:	100.00%	100	-	
Auto, Default and User Defined Bins:				
bin shift_rot_dir	98	1	-	Covered
bin <bins_bitwise[XOR_OP],dir_1>	16	1	-	Covered
bin <bins_bitwise[OR_OP],dir_1>	18	1	-	Covered
bin <bins_bitwise[XOR_OP],dir_0>	21	1	-	Covered
bin <bins_bitwise[OR_OP],dir_0>	17	1	-	Covered
bin <bins_arith[MUL_OP],dir_1>	14	1	-	Covered
bin <bins_arith[ADD_OP],dir_1>	21	1	-	Covered
bin <bins_arith[MUL_OP],dir_0>	22	1	-	Covered
bin <bins_arith[ADD_OP],dir_0>	18	1	-	Covered
Cross cross_shift_serial	100.00%	100	-	Covered
covered/total bins:	11	11	-	
missing/total bins:	0	11	-	
% Hit:	100.00%	100	-	
Auto, Default and User Defined Bins:				
bin shift_serial_in	48	1	-	Covered
bin <bins_bitwise[XOR_OP],serial_1>	14	1	-	Covered
bin <bins_bitwise[OR_OP],serial_1>	12	1	-	Covered
bin <bins_bitwise[XOR_OP],serial_0>	23	1	-	Covered
bin <bins_bitwise[OR_OP],serial_0>	23	1	-	Covered
bin <bins_arith[MUL_OP],serial_1>	16	1	-	Covered
bin <bins_arith[ADD_OP],serial_1>	20	1	-	Covered
bin <bins_arith[MUL_OP],serial_0>	20	1	-	Covered
bin <bins_arith[ADD_OP],serial_0>	19	1	-	Covered
bin <bins_shift[ROT_OP],serial_1>	25	1	-	Covered
bin <bins_shift[ROT_OP],serial_0>	25	1	-	Covered
Cross cross_redA_walking	95.23%	100	-	Uncovered
covered/total bins:	20	21	-	
missing/total bins:	1	21	-	
% Hit:	95.23%	100	-	
Auto, Default and User Defined Bins:				
bin redA_walking	7	1	-	Covered
bin <bins_bitwise[XOR_OP],walking_one[2],red_A_0>	3	1	-	Covered
bin <bins_bitwise[OR_OP],walking_one[2],red_A_0>	1	1	-	Covered
bin <bins_bitwise[XOR_OP],walking_one[1],red_A_0>	1	1	-	Covered

	1	1	-	Covered
bin <bins_bitwise[OR_OP],walking_one[1],red_A_0>	3	1	-	Covered
bin <bins_arith[MUL_OP],walking_one[2],red_A_1>	2	1	-	Covered
bin <bins_arith[ADD_OP],walking_one[2],red_A_1>	4	1	-	Covered
bin <bins_shift[ROT_OP],walking_one[2],red_A_1>	2	1	-	Covered
bin <bins_shift[SHIFT_OP],walking_one[2],red_A_1>	2	1	-	Covered
bin <bins_arith[MUL_OP],walking_one[2],red_A_0>	1	1	-	Covered
bin <bins_arith[ADD_OP],walking_one[2],red_A_0>	2	1	-	Covered
bin <bins_shift[ROT_OP],walking_one[2],red_A_0>	6	1	-	Covered
bin <bins_shift[SHIFT_OP],walking_one[2],red_A_0>	4	1	-	Covered
bin <bins_arith[MUL_OP],walking_one[1],red_A_1>	4	1	-	Covered
bin <bins_shift[ROT_OP],walking_one[1],red_A_1>	3	1	-	Covered
bin <bins_shift[SHIFT_OP],walking_one[1],red_A_1>	4	1	-	Covered
bin <bins_arith[MUL_OP],walking_one[1],red_A_0>	1	1	-	Covered
bin <bins_arith[ADD_OP],walking_one[1],red_A_0>	2	1	-	Covered
bin <bins_shift[ROT_OP],walking_one[1],red_A_0>	3	1	-	Covered
bin <bins_shift[SHIFT_OP],walking_one[1],red_A_0>	5	1	-	Covered
bin <bins_arith[ADD_OP],walking_one[1],red_A_1>	0	1	1	ZERO
Cross cross_redB_walking	90.47%	100	-	Uncovered
covered/total bins:	19	21	-	
missing/total bins:	2	21	-	
% Hit:	90.47%	100	-	
Auto, Default and User Defined Bins:				
bin redB_walking	13	1	-	Covered
bin <bins_bitwise[XOR_OP],walking_one[2],red_B_0>	2	1	-	Covered

bin <bins_bitwise[XOR_OP],walking_one[1],red_B_0>	3	1	-	Covered
bin <bins_bitwise[OR_OP],walking_one[1],red_B_0>	2	1	-	Covered
bin <bins_arith[ADD_OP],walking_one[2],red_B_1>	4	1	-	Covered
bin <bins_shift[ROT_OP],walking_one[2],red_B_1>	1	1	-	Covered
bin <bins_shift[SHIFT_OP],walking_one[2],red_B_1>	2	1	-	Covered
bin <bins_arith[MUL_OP],walking_one[2],red_B_0>	3	1	-	Covered
bin <bins_arith[ADD_OP],walking_one[2],red_B_0>	3	1	-	Covered
bin <bins_shift[ROT_OP],walking_one[2],red_B_0>	4	1	-	Covered
bin <bins_shift[SHIFT_OP],walking_one[2],red_B_0>	7	1	-	Covered
bin <bins_arith[MUL_OP],walking_one[1],red_B_1>	3	1	-	Covered
bin <bins_shift[ROT_OP],walking_one[1],red_B_1>	2	1	-	Covered
bin <bins_shift[SHIFT_OP],walking_one[1],red_B_1>	5	1	-	Covered
bin <bins_arith[MUL_OP],walking_one[1],red_B_0>	2	1	-	Covered
bin <bins_arith[ADD_OP],walking_one[1],red_B_0>	2	1	-	Covered
bin <bins_shift[ROT_OP],walking_one[1],red_B_0>	3	1	-	Covered
bin <bins_shift[SHIFT_OP],walking_one[1],red_B_0>	3	1	-	Covered
bin <bins_arith[MUL_OP],walking_one[2],red_B_1>	0	1	1	ZERO
bin <bins_arith[ADD_OP],walking_one[1],red_B_1>	0	1	1	ZERO
Cross cross_invalid_reduction	100.00%	100	-	Covered
covered/total bins:	13	13	-	
missing/total bins:	0	13	-	
% Hit:	100.00%	100	-	
Auto, Default and User Defined Bins:				
bin invalid_red	126	1	-	Covered
bin <bins_bitwise[XOR_OP],red_A_1,red_B_1>				

code_cvr_alsu - Notepad

File Edit Format View Help

bin <bins_bitwise[XOR_OP],red_A_1,red_B_1>	11	1	-	Covered
bin <bins_bitwise[XOR_OP],red_A_1,red_B_0>	10	1	-	Covered
bin <bins_bitwise[OR_OP],red_A_1,red_B_1>	9	1	-	Covered
bin <bins_bitwise[OR_OP],red_A_1,red_B_0>	10	1	-	Covered
bin <bins_bitwise[XOR_OP],red_A_0,red_B_1>	8	1	-	Covered
bin <bins_bitwise[XOR_OP],red_A_0,red_B_0>	8	1	-	Covered
bin <bins_bitwise[OR_OP],red_A_0,red_B_1>	6	1	-	Covered
bin <bins_bitwise[OR_OP],red_A_0,red_B_0>	10	1	-	Covered
bin <bins_arith[MUL_OP],red_A_0,red_B_0>	11	1	-	Covered
bin <bins_shift[ROT_OP],red_A_0,red_B_0>	15	1	-	Covered
bin <bins_arith[ADD_OP],red_A_0,red_B_0>	7	1	-	Covered
bin <bins_shift[SHIFT_OP],red_A_0,red_B_0>	14	1	-	Covered
Covergroup instance \alsu_pkg::alsu_txn::cvr_gp	100.00%	100	-	Covered
covered/total bins:	128	128	-	
missing/total bins:	0	128	-	
% Hit:	100.00%	100	-	
Coverpoint A_cp	100.00%	100	-	Covered
covered/total bins:	3	3	-	
missing/total bins:	0	3	-	
% Hit:	100.00%	100	-	
bin A_data_0	35	1	-	Covered
bin A_data_max	27	1	-	Covered
bin A_data_min	22	1	-	Covered
default bin A_data_default	163		-	Occurred
Coverpoint B_cp	100.00%	100	-	Covered
covered/total bins:	3	3	-	
missing/total bins:	0	3	-	
% Hit:	100.00%	100	-	
bin B_data_0	24	1	-	Covered
bin B_data_max	27	1	-	Covered
bin B_data_min	39	1	-	Covered

code_cvr_alsu - Notepad

File Edit Format View Help

bin B_data_min	39	1	-	Covered
default bin B_data_default	157		-	Occurred
Coverpoint opcode_cp	100.00%	100	-	Covered
covered/total bins:	6	6	-	
missing/total bins:	0	6	-	
% Hit:	100.00%	100	-	
illegal_bin bins_invalid	1		-	Occurred
bin bins_shift[SHIFT_OP]	48	1	-	Covered
bin bins_shift[ROT_OP]	50	1	-	Covered
bin bins_arith[ADD_OP]	39	1	-	Covered
bin bins_arith[MUL_OP]	36	1	-	Covered
bin bins_bitwise[OR_OP]	35	1	-	Covered
bin bins_bitwise[XOR_OP]	37	1	-	Covered
Coverpoint cin_cp	100.00%	100	-	Covered
covered/total bins:	2	2	-	
missing/total bins:	0	2	-	
% Hit:	100.00%	100	-	
bin cin_0	129	1	-	Covered
bin cin_1	117	1	-	Covered
Coverpoint direction_cp	100.00%	100	-	Covered
covered/total bins:	2	2	-	
missing/total bins:	0	2	-	
% Hit:	100.00%	100	-	
bin dir_0	126	1	-	Covered
bin dir_1	120	1	-	Covered
Coverpoint serial_in_cp	100.00%	100	-	Covered
covered/total bins:	2	2	-	
missing/total bins:	0	2	-	
% Hit:	100.00%	100	-	
bin serial_0	133	1	-	Covered
bin serial_1	113	1	-	Covered
Coverpoint red_op_A_cp	100.00%	100	-	Covered
covered/total bins:	2	2	-	
missing/total bins:	0	2	-	
% Hit:	100.00%	100	-	
bin red_A_0	117	1	-	Covered
bin red_A_1	129	1	-	Covered
Coverpoint red_op_B_cp	100.00%	100	-	Covered
covered/total bins:	2	2	-	
missing/total bins:	0	2	-	
% Hit:	100.00%	100	-	
bin red_B_0	127	1	-	Covered
bin red_B_1	119	1	-	Covered

bin red_B_1	119	1	-	Covered
Coverpoint A_walking_cp	100.00%	100	-	Covered
covered/total bins:	2	2	-	
missing/total bins:	0	2	-	
% Hit:	100.00%	100	-	
bin walking_one[1]	29	1	-	Covered
bin walking_one[2]	31	1	-	Covered
Coverpoint B_walking_cp	100.00%	100	-	Covered
covered/total bins:	2	2	-	
missing/total bins:	0	2	-	
% Hit:	100.00%	100	-	
bin walking_one[1]	32	1	-	Covered
bin walking_one[2]	36	1	-	Covered
Cross cross_arith_vals	100.00%	100	-	Covered
covered/total bins:	19	19	-	
missing/total bins:	0	19	-	
% Hit:	100.00%	100	-	
Auto, Default and User Defined Bins:				
bin add_mul_permutations	8	1	-	Covered
bin <A_data_min,B_data_min,bins_bitwise[XOR_OP]>	1	1	-	Covered
bin <A_data_min,B_data_min,bins_shift[ROT_OP]>	2	1	-	Covered
bin <A_data_min,B_data_min,bins_bitwise[OR_OP]>	1	1	-	Covered
bin <A_data_min,B_data_0,bins_shift[ROT_OP]>	1	1	-	Covered
bin <A_data_min,B_data_max,bins_bitwise[OR_OP]>	1	1	-	Covered
bin <A_data_min,B_data_0,bins_bitwise[OR_OP]>	1	1	-	Covered
bin <A_data_max,B_data_min,bins_shift[ROT_OP]>	2	1	-	Covered
bin <A_data_0,B_data_min,bins_shift[ROT_OP]>	2	1	-	Covered
bin <A_data_max,B_data_min,bins_bitwise[OR_OP]>	2	1	-	Covered
bin <A_data_0,B_data_min,bins_bitwise[OR_OP]>	1	1	-	Covered
bin <A_data_0,B_data_min,bins_shift[SHIFT_OP]>	1	1	-	Covered
bin <A_data_0,B_data_max,bins_bitwise[XOR_OP]>	1	1	-	Covered

bin <A_data_max,B_data_max,bins_shift[ROT_OP]>	1	1	-	Covered
bin <A_data_0,B_data_max,bins_shift[ROT_OP]>	1	1	-	Covered
bin <A_data_max,B_data_0,bins_shift[ROT_OP]>	2	1	-	Covered
bin <A_data_0,B_data_0,bins_bitwise[OR_OP]>	1	1	-	Covered
bin <A_data_0,B_data_max,bins_shift[SHIFT_OP]>	1	1	-	Covered
bin <A_data_max,B_data_0,bins_shift[SHIFT_OP]>	2	1	-	Covered
Cross cross_add_cin	100.00%	100	-	Covered
covered/total bins:	11	11	-	
missing/total bins:	0	11	-	
% Hit:	100.00%	100	-	
Auto, Default and User Defined Bins:				
bin add_cin_0_1	39	1	-	Covered
bin <bins_bitwise[XOR_OP],cin_1>	17	1	-	Covered
bin <bins_bitwise[OR_OP],cin_1>	19	1	-	Covered
bin <bins_bitwise[XOR_OP],cin_0>	20	1	-	Covered
bin <bins_bitwise[OR_OP],cin_0>	16	1	-	Covered
bin <bins_arith[MUL_OP],cin_1>	18	1	-	Covered
bin <bins_arith[MUL_OP],cin_0>	18	1	-	Covered
bin <bins_shift[ROT_OP],cin_1>	23	1	-	Covered
bin <bins_shift[SHIFT_OP],cin_1>	19	1	-	Covered
bin <bins_shift[ROT_OP],cin_0>	27	1	-	Covered
bin <bins_shift[SHIFT_OP],cin_0>	29	1	-	Covered
Cross cross_shift_rot_direction	100.00%	100	-	Covered
covered/total bins:	9	9	-	
missing/total bins:	0	9	-	
% Hit:	100.00%	100	-	
Auto, Default and User Defined Bins:				
bin shift_rot_dir	98	1	-	Covered
bin <bins_bitwise[XOR_OP],dir_1>	16	1	-	Covered
bin <bins_bitwise[OR_OP],dir_1>	18	1	-	Covered
bin <bins_bitwise[XOR_OP],dir_0>	21	1	-	Covered
bin <bins_bitwise[OR_OP],dir_0>	17	1	-	Covered
bin <bins_arith[MUL_OP],dir_1>	14	1	-	Covered
bin <bins_arith[ADD_OP],dir_1>	21	1	-	Covered
bin <bins_arith[MUL_OP],dir_0>	22	1	-	Covered
bin <bins_arith[ADD_OP],dir_0>	18	1	-	Covered

code_cvr_alsu - Notepad

File Edit Format View Help

bin <bins_arith[ADD_OP],dir_0>	18	1	-	Covered
Cross cross_shift_serial	100.00%	100	-	Covered
covered/total bins:	11	11	-	
missing/total bins:	0	11	-	
% Hit:	100.00%	100	-	
Auto, Default and User Defined Bins:				
bin shift_serial_in	48	1	-	Covered
bin <bins_bitwise[XOR_OP],serial_1>	14	1	-	Covered
bin <bins_bitwise[OR_OP],serial_1>	12	1	-	Covered
bin <bins_bitwise[XOR_OP],serial_0>	23	1	-	Covered
bin <bins_bitwise[OR_OP],serial_0>	23	1	-	Covered
bin <bins_arith[MUL_OP],serial_1>	16	1	-	Covered
bin <bins_arith[ADD_OP],serial_1>	20	1	-	Covered
bin <bins_arith[MUL_OP],serial_0>	20	1	-	Covered
bin <bins_arith[ADD_OP],serial_0>	19	1	-	Covered
bin <bins_shift[ROT_OP],serial_1>	25	1	-	Covered
bin <bins_shift[ROT_OP],serial_0>	25	1	-	Covered
Cross cross_redA_walking	100.00%	100	-	Covered
covered/total bins:	20	20	-	
missing/total bins:	0	20	-	
% Hit:	100.00%	100	-	
Auto, Default and User Defined Bins:				
bin redA_walking	7	1	-	Covered
bin <bins_bitwise[XOR_OP],walking_one[2],red_A_0>	3	1	-	Covered
bin <bins_bitwise[OR_OP],walking_one[2],red_A_0>	1	1	-	Covered
bin <bins_bitwise[XOR_OP],walking_one[1],red_A_0>	1	1	-	Covered
bin <bins_bitwise[OR_OP],walking_one[1],red_A_0>	3	1	-	Covered
bin <bins_arith[MUL_OP],walking_one[2],red_A_1>	2	1	-	Covered
bin <bins_arith[ADD_OP],walking_one[2],red_A_1>	4	1	-	Covered
bin <bins_shift[ROT_OP],walking_one[2],red_A_1>	2	1	-	Covered
bin <bins_shift[SHIFT_OP],walking_one[2],red_A_1>	2	1	-	Covered
bin <bins_arith[MUL_OP],walking_one[2],red_A_0>	1	1	-	Covered
bin <bins_arith[ADD_OP],walking_one[2],red_A_0>	2	1	-	Covered

bin <bins_shift[ROT_OP],walking_one[2],red_A_0>	2	1	-	Covered
bin <bins_shift[SHIFT_OP],walking_one[2],red_A_0>	6	1	-	Covered
bin <bins_arith[MUL_OP],walking_one[1],red_A_1>	4	1	-	Covered
bin <bins_shift[ROT_OP],walking_one[1],red_A_1>	4	1	-	Covered
bin <bins_shift[SHIFT_OP],walking_one[1],red_A_1>	3	1	-	Covered
bin <bins_arith[MUL_OP],walking_one[1],red_A_0>	4	1	-	Covered
bin <bins_arith[ADD_OP],walking_one[1],red_A_0>	1	1	-	Covered
bin <bins_shift[ROT_OP],walking_one[1],red_A_0>	2	1	-	Covered
bin <bins_shift[SHIFT_OP],walking_one[1],red_A_0>	3	1	-	Covered
Cross cross_redB_walking	5	1	-	Covered
covered/total bins:	100.00%	100	-	Covered
missing/total bins:	19	19	-	
% Hit:	0	19	-	
	100.00%	100	-	
Auto, Default and User Defined Bins:				
bin redB_walking	13	1	-	Covered
bin <bins_bitwise[XOR_OP],walking_one[2],red_B_0>	2	1	-	Covered
bin <bins_bitwise[OR_OP],walking_one[2],red_B_0>	3	1	-	Covered
bin <bins_bitwise[XOR_OP],walking_one[1],red_B_0>	3	1	-	Covered
bin <bins_bitwise[OR_OP],walking_one[1],red_B_0>	2	1	-	Covered
bin <bins_arith[ADD_OP],walking_one[2],red_B_1>	4	1	-	Covered
bin <bins_shift[ROT_OP],walking_one[2],red_B_1>	1	1	-	Covered
bin <bins_shift[SHIFT_OP],walking_one[2],red_B_1>	2	1	-	Covered
bin <bins_arith[MUL_OP],walking_one[2],red_B_0>	3	1	-	Covered
bin <bins_arith[ADD_OP],walking_one[2],red_B_0>	3	1	-	Covered
	3	1	-	Covered

	3	1	-	Covered
bin <bins_shift[ROT_OP],walking_one[2],red_B_0>	4	1	-	Covered
bin <bins_shift[SHIFT_OP],walking_one[2],red_B_0>	7	1	-	Covered
bin <bins_arith[MUL_OP],walking_one[1],red_B_1>	3	1	-	Covered
bin <bins_shift[ROT_OP],walking_one[1],red_B_1>	2	1	-	Covered
bin <bins_shift[SHIFT_OP],walking_one[1],red_B_1>	5	1	-	Covered
bin <bins_arith[MUL_OP],walking_one[1],red_B_0>	2	1	-	Covered
bin <bins_arith[ADD_OP],walking_one[1],red_B_0>	2	1	-	Covered
bin <bins_shift[ROT_OP],walking_one[1],red_B_0>	3	1	-	Covered
bin <bins_shift[SHIFT_OP],walking_one[1],red_B_0>	3	1	-	Covered
Cross cross_invalid_reduction	100.00%	100	-	Covered
covered/total bins:	13	13	-	
missing/total bins:	0	13	-	
% Hit:	100.00%	100	-	
Auto, Default and User Defined Bins:				
bin invalid_red	126	1	-	Covered
bin <bins_bitwise[XOR_OP],red_A_1,red_B_1>	11	1	-	Covered
bin <bins_bitwise[XOR_OP],red_A_1,red_B_0>	10	1	-	Covered
bin <bins_bitwise[OR_OP],red_A_1,red_B_1>	9	1	-	Covered
bin <bins_bitwise[OR_OP],red_A_1,red_B_0>	10	1	-	Covered
bin <bins_bitwise[XOR_OP],red_A_0,red_B_1>	8	1	-	Covered
bin <bins_bitwise[XOR_OP],red_A_0,red_B_0>	8	1	-	Covered
bin <bins_bitwise[OR_OP],red_A_0,red_B_1>	6	1	-	Covered
bin <bins_bitwise[OR_OP],red_A_0,red_B_0>	10	1	-	Covered
bin <bins_arith[MUL_OP],red_A_0,red_B_0>	11	1	-	Covered
bin <bins_arith[MUL_OP],red_A_0,red_B_0>	11	1	-	Covered
bin <bins_shift[ROT_OP],red_A_0,red_B_0>	15	1	-	Covered
bin <bins_arith[ADD_OP],red_A_0,red_B_0>	7	1	-	Covered
bin <bins_shift[SHIFT_OP],red_A_0,red_B_0>	14	1	-	Covered

out Coverage:

```

2  package alsu_pkg;
3      // ----- ENUM for opcodes -----
4      typedef enum logic [2:0] {
5          OR_OP      = 3'h0,
6          XOR_OP     = 3'h1,
7          ADD_OP     = 3'h2,
8          MUL_OP     = 3'h3,
9          SHIFT_OP   = 3'h4,
10         ROT_OP     = 3'h5,
11         INVALID_6  = 3'h6,
12         INVALID_7  = 3'h7
13     } opcode_e;
14
15     // ----- Transaction Class -----
16     class alsu_txn;
17
18         // Constants
19         localparam signed [2:0] MAXPOS = 3;
20         localparam signed [2:0] MAXNEG = -4;
21         localparam signed [2:0] ZERO   = 0;
22
23         // Randomizable Inputs
24         rand logic clk, rst, cin, serial_in;
25         rand logic red_op_A, red_op_B;
26         rand logic bypass_A, bypass_B, direction;
27         rand opcode_e opcode;
28         rand logic signed [2:0] A, B;
29
30         // Fixed array for constraint #8
31         rand opcode_e opcode_seq[6];
32
33         // Constraint enable flags
34         bit en_c1, en_c2, en_c3, en_c4, en_c5, en_c6, en_c7, en_c8;
35

```

```

36 // ----- Covergroup -----
37 covergroup cvr_gp @(posedge clk);
38     option.per_instance = 1;
39
40     // ----- Base Coverpoints -----
41     A_cp: coverpoint A {
42         bins A_data_0      = {ZERO};
43         bins A_data_max    = {MAXPOS};
44         bins A_data_min    = {MAXNEG};
45         bins A_data_default = default;
46     }
47
48     B_cp: coverpoint B {
49         bins B_data_0      = {ZERO};
50         bins B_data_max    = {MAXPOS};
51         bins B_data_min    = {MAXNEG};
52         bins B_data_default = default;
53     }
54
55     opcode_cp: coverpoint opcode {
56         bins bins_shift[]  = {SHIFT_OP, ROT_OP};
57         bins bins_arith[]  = {ADD_OP, MUL_OP};
58         bins bins_bitwise[] = {OR_OP, XOR_OP};
59         illegal_bins bins_invalid = {INVALID_6, INVALID_7};
60     }
61
62     cin_cp: coverpoint cin {
63         bins cin_0 = {0};
64         bins cin_1 = {1};
65     }
66

```

```
67 direction_cp: coverpoint direction {
68     bins dir_0 = {0};
69     bins dir_1 = {1};
70 }
71
72 serial_in_cp: coverpoint serial_in {
73     bins serial_0 = {0};
74     bins serial_1 = {1};
75 }
76
77 red_op_A_cp: coverpoint red_op_A {
78     bins red_A_0 = {0};
79     bins red_A_1 = {1};
80 }
81
82 red_op_B_cp: coverpoint red_op_B {
83     bins red_B_0 = {0};
84     bins red_B_1 = {1};
85 }
86
87 // Walking-one patterns for A
88 A_walking_cp: coverpoint A {
89     bins walking_one[] = {3'b001, 3'b010, 3'b100};
90 }
91
92 // Walking-one patterns for B
93 B_walking_cp: coverpoint B {
94     bins walking_one[] = {3'b001, 3'b010, 3'b100};
95 }
96
```

```

97 // ----- Cross Coverage -----
98 // 1) ADD/MUL: permutations of A and B with {MAXPOS, MAXNEG, ZERO}
99 cross_arith_vals: cross A_cp, B_cp, opcode_cp {
100   bins add_mul_permutations = binsof(opcode_cp.bins_arith) &&
101   | (binsof(A_cp.A_data_0) || binsof(A_cp.A_data_max) || binsof(A_cp.A_data_min)) &&
102   | (binsof(B_cp.B_data_0) || binsof(B_cp.B_data_max) || binsof(B_cp.B_data_min));
103 }
104
105 // 2) When ADD, cin should take 0 and 1
106 cross_add_cin: cross opcode_cp, cin_cp {
107   bins add_cin_0_1 = binsof(opcode_cp) intersect {ADD_OP};
108 }
109
110 // 3) When SHIFT/ROT, direction should take 0 and 1
111 cross_shift_rot_direction: cross opcode_cp, direction_cp {
112   bins shift_rot_dir = binsof(opcode_cp.bins_shift);
113 }
114
115 // 4) When SHIFT, serial_in should take 0 and 1
116 cross_shift_serial: cross opcode_cp, serial_in_cp {
117   bins shift_serial_in = binsof(opcode_cp) intersect {SHIFT_OP};
118 }
119
120 // 5) OR/XOR + red_op_A asserted: A walking-one patterns, B == 0
121 cross_redA_walking: cross opcode_cp, A_walking_cp, red_op_A_cp {
122   bins redA_walking = binsof(opcode_cp.bins_bitwise) &&
123   | binsof(red_op_A_cp.red_A_1);
124 }
125
126 // 6) OR/XOR + red_op_B asserted: B walking-one patterns, A == 0
127 cross_redB_walking: cross opcode_cp, B_walking_cp, red_op_B_cp {
128   bins redB_walking = binsof(opcode_cp.bins_bitwise) &&
129   | binsof(red_op_B_cp.red_B_1);
130 }
131
132 // 7) Invalid reduction case
133 cross_invalid_reduction: cross opcode_cp, red_op_A_cp, red_op_B_cp {
134   bins invalid_red = (binsof(opcode_cp) intersect {ADD_OP, MUL_OP, SHIFT_OP, ROT_OP}) &&
135   | (binsof(red_op_A_cp.red_A_1) || binsof(red_op_B_cp.red_B_1));
136 }
137
138 endgroup
139
140 // ----- Constraints -----
141 constraint c1 { if (en_c1) A inside {[MAXNEG:MAXPOS]}; }
142 constraint c2 { if (en_c2) B inside {[MAXNEG:MAXPOS]}; }
143 constraint c3 { if (en_c3) opcode inside {OR_OP, XOR_OP, ADD_OP, MUL_OP, SHIFT_OP, ROT_OP}; }
144
145 // Constraint #8: unique opcodes in array
146 constraint unique_opcodes {
147   if (en_c8) {
148     unique {opcode_seq};
149     foreach (opcode_seq[i]) {
150       opcode_seq[i] inside {OR_OP, XOR_OP, ADD_OP, MUL_OP, SHIFT_OP, ROT_OP};
151     }
152   }
153 }
154

```

```

155 // Walking-one patterns constraint
156 constraint walking_one_constraint {
157     if ((opcode inside {OR_OP, XOR_OP}) && red_op_A && (B == 0)) {
158         A inside {3'b001, 3'b010, 3'b100};
159     }
160     if ((opcode inside {OR_OP, XOR_OP}) && red_op_B && (A == 0)) {
161         B inside {3'b001, 3'b010, 3'b100};
162     }
163 }
164
165 // ----- Constructor -----
166 function new();
167     en_c1 = 1; en_c2 = 1; en_c3 = 1;
168     en_c4 = 1; en_c5 = 1; en_c6 = 1;
169     en_c7 = 1; en_c8 = 0;
170     cvr_gp = new();
171 endfunction
172
173 endclass : alsu_txn
174
175 endpackage : alsu_pkg

```

```

2  import alsu_pkg::*;
3
4  module question_3_tb;
5
6      // DUT Signals
7      logic clk, rst;
8      logic signed [2:0] A, B;
9      logic cin, serial_in;
10     logic red_op_A, red_op_B;
11     logic bypass_A, bypass_B, direction;
12     opcode_e opcode;
13     logic signed [5:0] result;
14     logic [15:0] leds;
15
16     // Transaction object
17     alsu_txn t;
18
19     // Instantiate DUT
20     ALSU DUT (
21         .clk(clk),
22         .rst(rst),
23         .A(A),
24         .B(B),
25         .cin(cin),
26         .serial_in(serial_in),
27         .red_op_A(red_op_A),
28         .red_op_B(red_op_B),
29         .bypass_A(bypass_A),
30         .bypass_B(bypass_B),
31         .direction(direction),
32         .opcode(opcode),
33         .out(result),
34         .leds(leds)
35     );
36

```

```

37 // Clock generation
38 always #5 clk = ~clk;
39
40 // Helper arrays
41 logic signed [2:0] A_vals[3] = '{ -4, 0, 3 }; // {MAXNEG, ZERO, MAXPOS}
42 logic signed [2:0] B_vals[3] = '{ -4, 0, 3 };
43
44 logic direction_val[2] = '{0,1};
45 logic serial_in_val[2] = '{0,1};
46
47 // Covergroup sampling process
48 initial begin
49     forever begin
50         @(posedge clk);
51         if (!rst && !bypass_A && !bypass_B) begin
52             t.cvr_gp.sample();
53         end
54     end
55 end
56
57 // ----- Assertions -----
58 // Assert A walking-one
59 assert property (@(posedge clk) disable iff (rst)
60     (opcode inside {OR_OP, XOR_OP} && red_op_A && (B == 0)) |-> (A inside {3'b001, 3'b010, 3'b100}))
61     else $error("A is not walking-one when red_op_A is active!");
62
63 // Assert B walking-one
64 assert property (@(posedge clk) disable iff (rst)
65     (opcode inside {OR_OP, XOR_OP} && red_op_B && (A == 0)) |-> (B inside {3'b001, 3'b010, 3'b100}))
66     else $error("B is not walking-one when red_op_B is active!");
67

```

```

68 // Main test sequence
69 initial begin
70     clk = 0;
71     rst = 1;
72     #20 rst = 0;
73
74     t = new();
75
76     // Initialize defaults
77     bypass_A = 0;
78     bypass_B = 0;
79     red_op_A = 0;
80     red_op_B = 0;
81     cin = 0;
82     serial_in = 0;
83     direction = 0;
84
85     // -----
86     // STAGE 1: DIRECTED TESTS
87     // -----
88     $display("Starting directed tests...");
89
90     // 1) ADD & MUL with A,B permutations
91     foreach (A_vals[i]) begin
92         foreach (B_vals[j]) begin
93             A = A_vals[i];
94             B = B_vals[j];
95
96             @(posedge clk);
97             opcode = ADD_OP;
98             cin = 0; @(posedge clk);
99             cin = 1; @(posedge clk);
100

```

```

101         opcode = MUL_OP;
102         @(posedge clk);
103     end
104 end
105
106 // 2) direction and serial_in tests
107 foreach (direction_val[d]) begin
108     direction = direction_val[d];
109
110     opcode = SHIFT_OP;
111     foreach (serial_in_val[s]) begin
112         serial_in = serial_in_val[s];
113         @(posedge clk);
114     end
115
116     opcode = ROT_OP;
117     @(posedge clk);
118 end
119
120 // 3) Walking-One A patterns
121 red_op_A = 1;
122 B = 0;
123 for (int k = 0; k < 3; k++) begin
124     A = (3'b001 << k);
125     opcode = OR_OP; @(posedge clk);
126     opcode = XOR_OP; @(posedge clk);
127 end
128 red_op_A = 0;
129

```

```

130 // 4) Walking-One B patterns
131 red_op_B = 1;
132 A = 0;
133 for (int k = 0; k < 3; k++) begin
134     B = (3'b001 << k);
135     opcode = OR_OP; @(posedge clk);
136     opcode = XOR_OP; @(posedge clk);
137 end
138 red_op_B = 0;
139
140 // 5) Invalid Reduction cases
141 red_op_A = 1;
142 opcode = ADD_OP; @(posedge clk);
143 opcode = MUL_OP; @(posedge clk);
144 red_op_A = 0;
145
146 red_op_B = 1;
147 opcode = SHIFT_OP; @(posedge clk);
148 opcode = ROT_OP; @(posedge clk);
149 red_op_B = 0;
150
151 $display("Directed tests completed.");
152
153 // -----
154 // STAGE 2: RANDOMIZED TESTS
155 // -----
156 $display("Starting random tests...");
157
158 t.en_c8 = 0;
159 repeat (1000) begin
160     if (!t.randomize()) begin
161         $display("Randomization failed!");
162         continue;

```

```

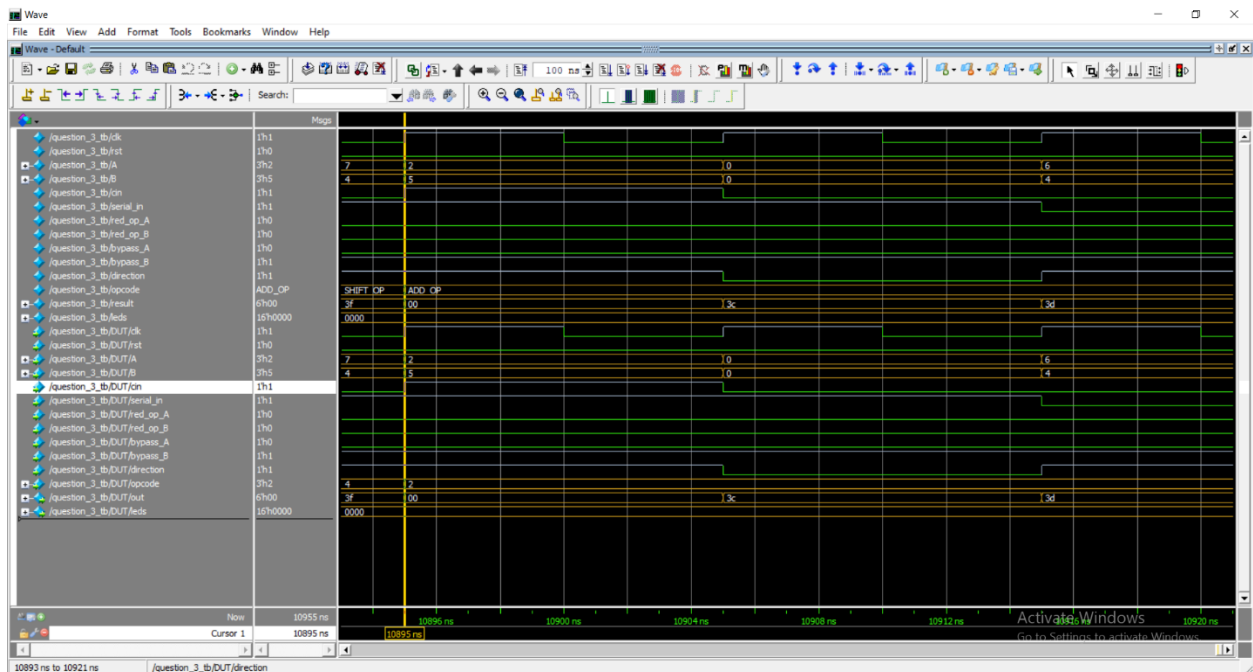
163     end
164
165     A = t.A;
166     B = t.B;
167     cin = t.cin;
168     serial_in = t.serial_in;
169     red_op_A = t.red_op_A;
170     red_op_B = t.red_op_B;
171     bypass_A = t.bypass_A;
172     bypass_B = t.bypass_B;
173     direction = t.direction;
174     opcode = t.opcode;
175
176     @(posedge clk);
177 end
178
179 // -----
180 // STAGE 3: UNIQUE OPCODE SEQUENCE
181 // -----
182 $display("Starting unique opcode sequence loop...");
183
184 t.en_c1 = 0;
185 t.en_c2 = 0;
186 t.en_c3 = 0;
187 t.en_c8 = 1;
188
189 if (!t.randomize()) begin
190     $display("Randomization failed for opcode sequence!");
191 end else begin
192     foreach (t.opcode_seq[i]) begin
193         opcode = t.opcode_seq[i];
194

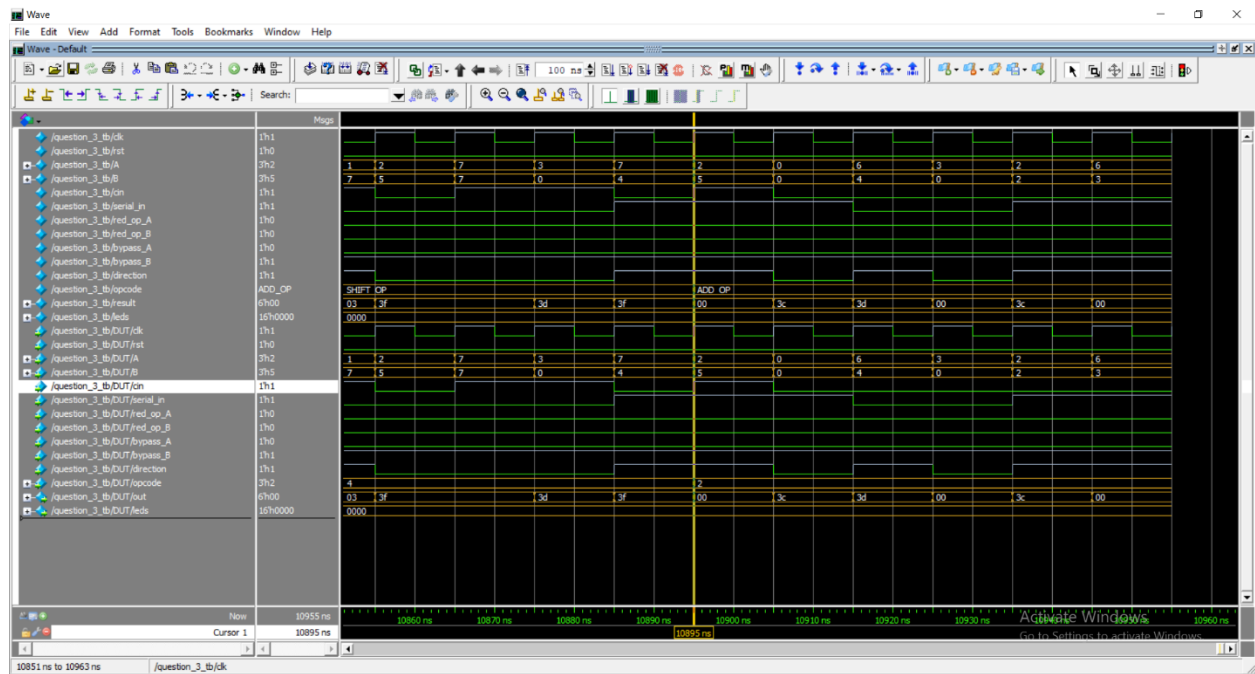
```

```

194
195     repeat (6) begin
196         if (!t.randomize()) begin
197             $display("Randomization failed inside unique opcode loop!");
198             continue;
199         end
200
201         A = t.A;
202         B = t.B;
203         cin = t.cin;
204         serial_in = t.serial_in;
205         direction = t.direction;
206
207         @(posedge clk);
208     end
209 end
210 end
211
212 $display("Simulation completed.");
213 $display("Coverage: %.2f%%", t.cvr_gp.get_inst_coverage());
214 $finish;
215 end
216
217 // Monitor to display transactions
218 initial begin
219     $monitor("Time: %t | Opcode: %s | A: %0d | B: %0d | Result: %0d | LEDs: %h",
220             $time, opcode.name(), A, B, result, leds);
221 end
222
223 endmodule
224

```





Q2

```

# Starting directed assertion tests...
# Testing Assertion 4-i: Decoder one-hot
# Testing Assertion 4-ii: Priority encoder valid
# Testing Assertion 1: a -> ##2 b
# Testing Assertion 2: (a && b) -> ##[1:3] c
# Testing Assertion 3: sequence s11b (##2 !b)
# Starting 10,000 random test iterations...
# Completed 0 random tests...
# Completed 1000 random tests...
# Completed 2000 random tests...
# Completed 3000 random tests...
# Completed 4000 random tests...
# Completed 5000 random tests...
# Completed 6000 random tests...
# Completed 7000 random tests...
# Completed 8000 random tests...
# Completed 9000 random tests...
# 10,000 test iterations completed successfully!
# All directed assertion tests should have passed.

```

ASSERTION RESULTS:

Name	File(Line)	Failure Count	Pass Count
/tb_complete/assert__priority_encoder_valid	tb_complete.svh(57)	0	1
/tb_complete/assert__decoder_one_hot_delayed	tb_complete.svh(54)	0	1
/tb_complete/assert__prop_s11b	tb_complete.svh(47)	0	1
/tb_complete/assert__a_and_b_then_c_1to3	tb_complete.svh(44)	0	1
/tb_complete/assert__a_then_b_after_2	tb_complete.svh(41)	0	1

ass_cvr - Notepad

File Edit Format View Help

=== Instance: /tb_complete/dut_priority

=== Design Unit: work.priority_enc

Branch Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
Branches	7	7	0	100.00%

=====Branch Details=====

Branch Coverage for instance /tb_complete/dut_priority

Line	Item	Count	Source
File priority_enc.v			
-----IF Branch-----			
10		9806	Count coming in to IF
10	1	2	if (rst) begin
14	1	9804	else begin

Branch totals: 2 hits of 2 branches = 100.00%

-----CASE Branch-----			
15		9804	Count coming in to CASE
16	1	4830	4'b1xxx: Y <= 2'b11; // Highest priority: D[3]
17	1	2476	4'b01xx: Y <= 2'b10; // D[2]
18	1	1280	4'b001x: Y <= 2'b01; // D[1]
19	1	628	4'b0001: Y <= 2'b00; // D[0]
20	1	590	default: Y <= 2'b00; // Handle all other cases

Branch totals: 5 hits of 5 branches = 100.00%

Expression Coverage:

Enabled Coverage	Bins	Covered	Misses	Coverage
Expressions	1	1	0	100.00%

=====Expression Details=====

Expression Coverage for instance /tb_complete/dut_priority --

File priority_enc.v

-----Focused Expression View-----

Line 23 Item 1 (D != 0)

Expression totals: 1 of 1 input term covered = 100.00%

Input Term	Covered	Reason for no coverage	Hint
(D != 0)	Y		

Rows:	Hits	FEC Target	Non-masking condition(s)
Row 1:	1	(D != 0)_0	-
Row 2:	1	(D != 0)_1	-

Statement Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
Statements	9	9	0	100.00%

=====Statement Details=====

Statement Coverage for instance /tb_complete/dut_priority --

Line	Item	Count	Source
----	----	----	-----
File priority_enc.v			
1			module priority_enc (
2			input clk,
3			input rst,
4			input [3:0] D,
5			output reg [1:0] Y,
6			output reg valid

```

5             output reg [1:0] Y,
6             | output reg valid
7             );
8
9             1           9806 always @(posedge clk) begin
10
11                 1           2             Y <= 2'b00;
12                 1           2             valid <= 1'b0;
13
14                 end
15
16                 else begin
17                     casex (D)
18                         1           4830             4'b1xxx: Y <= 2'b11; // Highest priority: D[3]
19                         1           2476             4'b01xx: Y <= 2'b10; // D[2]
20                         1           1280             4'b001x: Y <= 2'b01; // D[1]
21                         1           628              4'b0001: Y <= 2'b00; // D[0]
22                         1           590              default: Y <= 2'b00; // Handle all other cases
23
24                     endcase
25
26                 valid <= (D != 4'b0000);

```

ass_cvr - Notepad

File Edit Format View Help

Toggle Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Toggles	16	16	0	100.00%

=====Toggle Details=====

Toggle Coverage for instance /tb_complete/dut_priority --

Node	1H->0L	0L->1H	"Coverage"
D[0-3]	1	1	100.00
Y[1-0]	1	1	100.00
clk	1	1	100.00
valid	1	1	100.00

Total Node Count = 8
 Toggled Node Count = 8
 Untoggled Node Count = 0

Toggle Coverage = 100.00% (16 of 16 bins)

ass_cvr - Notepad

File Edit Format View Help

=== Instance: /tb_complete/dut_decoder

=== Design Unit: work.decoder_3to8

Branch Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Branches	10	10	0	100.00%

=====Branch Details=====

Branch Coverage for instance /tb_complete/dut_decoder

Line	Item	Count	Source
----	----	----	-----
File decoder_3to8.v			
-----IF Branch-----			
9		9854	Count coming in to IF
9	1	2	if (rst) begin
12	1	9852	else begin
-----CASE Branch-----			
13		9852	Count coming in to CASE
14	1	1183	3'b000: Y <= 8'b00000001;
15	1	1271	3'b001: Y <= 8'b00000010;
16	1	1257	3'b010: Y <= 8'b00000100;
17	1	1240	3'b011: Y <= 8'b00001000;
18	1	1212	3'b100: Y <= 8'b00010000;
19	1	1252	3'b101: Y <= 8'b00100000;
20	1	1235	3'b110: Y <= 8'b01000000;
21	1	1202	3'b111: Y <= 8'b10000000;

Branch totals: 8 hits of 8 branches = 100.00%

Statement Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Statements	10	10	0	100.00%

=====Statement Details=====

Statement Coverage for instance /tb_complete/dut_decoder --

Line	Item	Count	Source
----	----	----	-----
File decoder_3to8.v			
1			module decoder_3to8 (
2			input clk,
3			input rst,
4			input [2:0] in,
5			output reg [7:0] Y
6			);
7			
8	1	9854	always @(posedge clk or posedge rst) begin
9			if (rst) begin
10	1	2	Y <= 8'b00000000;
11			end
12			else begin
13			case (in)
14	1	1183	3'b000: Y <= 8'b00000001;
15	1	1271	3'b001: Y <= 8'b00000010;

ass_cvr - Notepad

File Edit Format View Help

15	1	1271	3'b001: Y <= 8'b00000010;
16	1	1257	3'b010: Y <= 8'b00000100;
17	1	1240	3'b011: Y <= 8'b00001000;
18	1	1212	3'b100: Y <= 8'b00010000;
19	1	1252	3'b101: Y <= 8'b00100000;
20	1	1235	3'b110: Y <= 8'b01000000;
21	1	1202	3'b111: Y <= 8'b10000000;

Toggle Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	----	-----
Toggles	24	24	0	100.00%

=====Toggle Details=====

Toggle Coverage for instance /tb_complete/dut_decoder --

Node	1H->0L	0L->1H	"Coverage"
-----	-----	-----	-----
Y[7-0]	1	1	100.00
clk	1	1	100.00
in[0-2]	1	1	100.00

Total Node Count = 12
Toggled Node Count = 12
Untoggled Node Count = 0

Toggle Coverage = 100.00% (24 of 24 bins)

```

1  module priority_enc (
2      input  clk,
3      input  rst,
4      input  [3:0] D,
5      output reg [1:0] Y,
6      output reg valid
7  );
8
9  always @(posedge clk) begin
10     if (rst) begin
11         Y <= 2'b00;
12         valid <= 1'b0;
13     end
14     else begin
15         casex (D)
16             4'b1xxx: Y <= 2'b11; // Highest priority: D[3]
17             4'b01xx: Y <= 2'b10; // D[2]
18             4'b001x: Y <= 2'b01; // D[1]
19             4'b0001: Y <= 2'b00; // D[0]
20             default: Y <= 2'b00; // Handle all other cases
21         endcase
22
23         valid <= (D != 4'b0000);
24     end
25 end
26 endmodule

```

```

1  module decoder_3to8 (
2      input clk,
3      input rst,
4      input [2:0] in,
5      output reg [7:0] Y
6  );
7
8  always @(posedge clk or posedge rst) begin
9      if (rst) begin
10         Y <= 8'b00000000;
11     end
12     else begin
13         case (in)
14             3'b000: Y <= 8'b00000001;
15             3'b001: Y <= 8'b00000010;
16             3'b010: Y <= 8'b00000100;
17             3'b011: Y <= 8'b00001000;
18             3'b100: Y <= 8'b00010000;
19             3'b101: Y <= 8'b00100000;
20             3'b110: Y <= 8'b01000000;
21             3'b111: Y <= 8'b10000000;
22             default: Y <= 8'b00000000;
23         endcase
24     end
25 end
26 endmodule

```

```

1  vlib work
2  vlog priority_enc.v decoder_3to8.v assertions_module.svh tb_complete.svh +cover -covercells
3  vsim -voptargs=+acc work.tb_complete -cover
4  add wave *
5  coverage save tb_complete.ucdb -onexit
6  run -all

```

```

1  package assertions_pkg;
2
3  // Property for assertion 1: a -> ##2 b
4  property a_then_b_after_2(clk, rst, test_en, a, b);
5      @(posedge clk) disable iff (rst) (test_en && a) |-> ##2 b;
6  endproperty
7
8
9  // Property for assertion 2: (a && b) -> ##[1:3] c
10 property a_and_b_then_c_1to3(clk, rst, test_en, a, b, c);
11     @(posedge clk) disable iff (rst)
12     $rose(test_en && a && b) |-> ##[1:3] c;
13 endproperty
14
15 // Sequence for assertion 3: ##2 !b
16 sequence s11b(b);
17     ##2 !b;
18 endsequence
19
20 property prop_s11b(clk, rst, test_en, b);
21     @(posedge clk) disable iff (rst) test_en |-> s11b(b);
22 endproperty
23
24 // Property for assertion 4-i: Decoder one-hot
25 property decoder_one_hot(clk, rst, decoder_Y);
26     @(posedge clk) disable iff (rst) $onehot(decoder_Y);
27 endproperty
28
29 // Property for assertion 4-ii: Priority encoder valid
30 property priority_encoder_valid(clk, rst, D, valid);
31     @(posedge clk) disable iff (rst) (D == 4'b0000) |> !valid;
32 endproperty
33
34 endpackage

```

```
1  import assertions_pkg::*;
2
3  module tb_complete();
4      logic clk, rst;
5      logic a, b, c;
6
7      // Priority Encoder signals
8      logic [3:0] D;
9      logic [1:0] Y;
10     logic valid;
11
12     // 3-to-8 Decoder signals
13     logic [2:0] decoder_in;
14     logic [7:0] decoder_Y;
15
16     // Test control flags
17     logic test_assertion_1;
18     logic test_assertion_2;
19     logic test_assertion_3;
20
21     // Instantiate modules
22     priority_enc dut_priority (
23         .clk(clk),
24         .rst(rst),
25         .D(D),
26         .Y(Y),
27         .valid(valid)
28     );
29
30     decoder_3to8 dut_decoder (
31         .clk(clk),
32         .rst(rst),
33         .in(decoder_in),
34         .Y(decoder_Y)
```

```

34     .Y(decoder_Y)
35 );
36
37 // Clock generation
38 always #5 clk = ~clk;
39
40 // Assertions - ADD DELAY FOR DECODER
41 assert property (a_then_b_after_2(clk, rst, test_assertion_1, a, b))
42     else $error("Assertion 1 Failed: 'a' was high, but 'b' did not go high after 2 cycles");
43
44 assert property (a_and_b_then_c_1to3(clk, rst, test_assertion_2, a, b, c))
45     else $error("Assertion 2 Failed: 'a && b' were high, but 'c' was not high within 1 to 3 cycles");
46
47 assert property (prop_s11b(clk, rst, test_assertion_3, b))
48     else $error("Assertion 3 Failed: Sequence s11b - 'b' was not low after 2 cycles");
49
50 // Add small delay for decoder to stabilize
51 property decoder_one_hot_delayed;
52     @(posedge clk) disable iff (rst) ##1 $onehot(decoder_Y);
53 endproperty
54 assert property (decoder_one_hot_delayed)
55     else $error("Assertion 4-i Failed: 'decoder_Y' is not one-hot encoded!");
56
57 assert property (priority_encoder_valid(clk, rst, D, valid))
58     else $error("Assertion 4-ii Failed: All D inputs were 0, but 'valid' was not low in the next cycle");
59
60 // Test sequence
61 initial begin
62     integer i;
63
64     // Initialize
65     clk = 0;
66     rst = 1;
67     a = 0; b = 0; c = 0;

```

```

68     D = 4'b0000;
69     decoder_in = 3'b000;
70     test_assertion_1 = 0;
71     test_assertion_2 = 0;
72     test_assertion_3 = 0;
73
74     // Release reset
75     #20 rst = 0;
76     @(posedge clk); // Wait for first clock after reset
77
78     $display("Starting directed assertion tests...");
79
80     // ===== TEST ASSERTION 4-i FIRST =====
81     $display("Testing Assertion 4-i: Decoder one-hot");
82     // Test all decoder inputs first to avoid false failures
83     for (i = 0; i < 8; i++) begin
84         decoder_in <= i;
85         @(posedge clk);
86         // Wait for decoder to stabilize
87         #1; // Small delay after clock edge
88     end
89     decoder_in <= 3'b000;
90     @(posedge clk);
91     #1; // Small delay after clock edge
92
93     // ===== TEST ASSERTION 4-ii =====
94     $display("Testing Assertion 4-ii: Priority encoder valid");
95     D <= 4'b0000;
96     @(posedge clk); // valid should be low next cycle
97     @(posedge clk); // Check
98

```

```
99 // ===== TEST ASSERTION 1 =====
100 $display("Testing Assertion 1: a -> ##2 b");
101 test_assertion_1 = 1;
102
103 // Setup: a goes high
104 a <= 1;
105 @(posedge clk);
106 a <= 0;
107
108 // Wait 1 cycle
109 @(posedge clk);
110
111 // Set b high at cycle 2 (should pass)
112 b <= 1;
113 @(posedge clk);
114 b <= 0;
115
116 test_assertion_1 = 0;
117 @(posedge clk);
118 @(posedge clk); // Extra cycle to ensure assertion completes
119
120 // ===== TEST ASSERTION 2 =====
121 $display("Testing Assertion 2: (a && b) -> ##[1:3] c");
122 test_assertion_2 = 1;
123
124 // Setup: a and b go high
125 a <= 1; b <= 1;
126 @(posedge clk);
127
128 // Set c high at cycle 1 (should pass)
129 c <= 1;
130 @(posedge clk);
131 a <= 0; b <= 0; c <= 0;
```

```

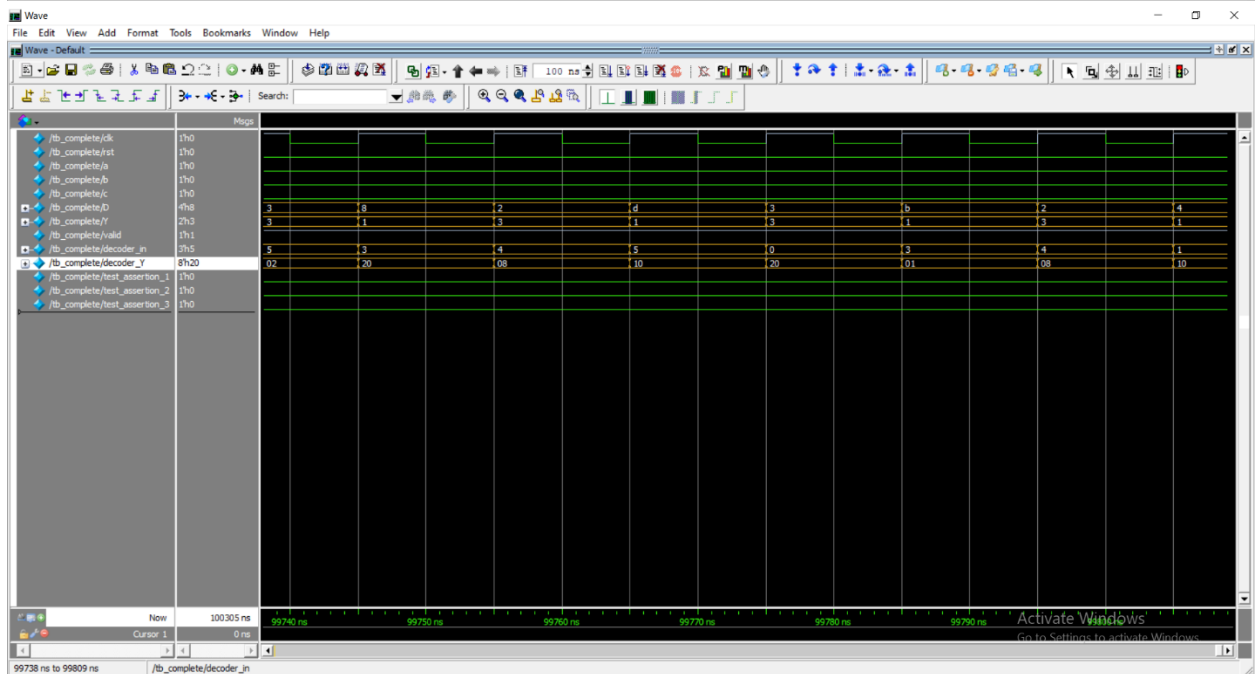
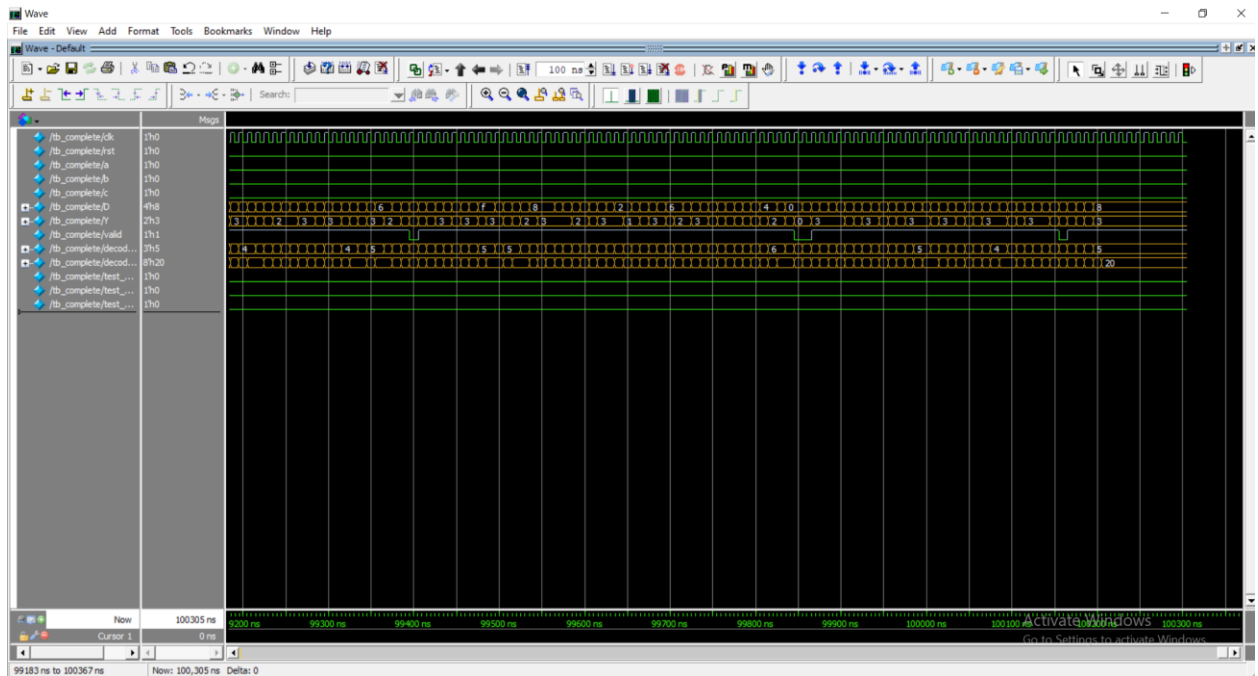
127
128 // Set c high at cycle 1 (should pass)
129 c <= 1;
130 @(posedge clk);
131 a <= 0; b <= 0; c <= 0;
132 @(posedge clk); // Extra cycle
133
134 test_assertion_2 = 0;
135 @(posedge clk);
136
137 // ===== TEST ASSERTION 3 =====
138 $display("Testing Assertion 3: sequence s11b (##2 !b)");
139 test_assertion_3 = 1;
140
141 // Setup: b goes high
142 b <= 1;
143 @(posedge clk);
144 @(posedge clk);
145
146 // Set b low at cycle 2 (should pass)
147 b <= 0;
148 @(posedge clk);
149
150 test_assertion_3 = 0;
151 @(posedge clk);
152 @(posedge clk); // Extra cycle to ensure assertion completes
153
154 // ===== RANDOM TESTS =====
155 $display("Starting 10,000 random test iterations...");
156
157 // Disable assertion testing during random tests
158 test_assertion_1 = 0;
159 test_assertion_2 = 0;
160 test_assertion_3 = 0;

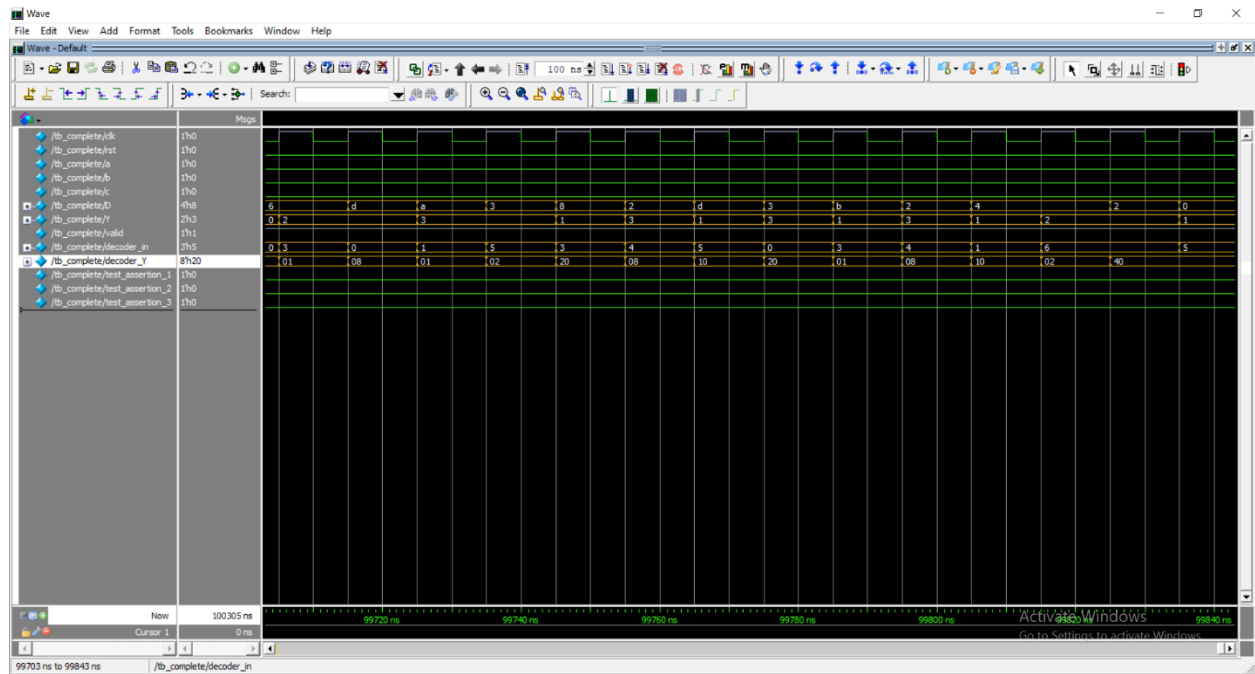
```

```

156
157     // Disable assertion testing during random tests
158     test_assertion_1 = 0;
159     test_assertion_2 = 0;
160     test_assertion_3 = 0;
161
162     // Main repeat loop with 10,000 iterations
163     for (i = 0; i < 10000; i++) begin
164         // Randomize all inputs
165         D <= $urandom;
166         decoder_in <= $urandom_range(0, 7);
167         a <= 0; // Keep assertion signals low during random tests
168         b <= 0;
169         c <= 0;
170
171         @(posedge clk);
172
173         // Display progress every 1000 tests
174         if (i % 1000 == 0) begin
175             $display("Completed %0d random tests...", i);
176         end
177     end
178
179     $display("10,000 test iterations completed successfully!");
180     $display("All directed assertion tests should have passed.");
181     #100 $finish;
182 end
183
184 endmodule

```





Q3

Activate Windows

Go to Settings to activate Windows.

DIRECTIVE COVERAGE:

Name	Design Unit	Design UnitType	Lang	File(Line)	Hits	Status
/counter_top/dut/binding_inst/c_load	counter_sva	Verilog	SVA	counter_sva.sv(9)	3	Covered
/counter_top/dut/binding_inst/c_increment	counter_sva	Verilog	SVA	counter_sva.sv(32)	6	Covered
/counter_top/dut/binding_inst/c_decrement	counter_sva	Verilog	SVA	counter_sva.sv(41)	6	Covered
/counter_top/dut/binding_inst/c_max_count	counter_sva	Verilog	SVA	counter_sva.sv(56)	14	Covered
/counter_top/dut/binding_inst/c_zero	counter_sva	Verilog	SVA	counter_sva.sv(64)	6	Covered

TOTAL DIRECTIVE COVERAGE: 100.00% COVERS: 5

ASSERTION RESULTS:

Name	File(Line)	Failure Count	Pass Count
/counter_top/dut/binding_inst/a_load	counter_sva.sv(8)	0	1
/counter_top/dut/binding_inst/a_increment	counter_sva.sv(31)	0	1
/counter_top/dut/binding_inst/a_decrement	counter_sva.sv(40)	0	1
/counter_top/dut/binding_inst/a_reset	counter_sva.sv(46)	0	1
/counter_top/dut/binding_inst/a_max_count	counter_sva.sv(55)	0	1
/counter_top/dut/binding_inst/a_zero	counter_sva.sv(63)	0	1

Name	Assertion Type	Language	Enable	Failure Count	Pass Count	Active Count	Memory	Peak Memory	Peak Memory Time	Cumulative Threads	ATV	Assertion Expression	Included
/counter_top/iva_inst/assert_0	Concurrent	SVA	on	0	0	-	0B	0B	0 ns	0	off	assert(@(posedge vif.clk) disable i...	✓
/counter_top/iva_inst/assert_1	Concurrent	SVA	on	0	0	-	0B	0B	0 ns	0	off	assert(@(posedge vif.clk) disable i...	✓
/counter_top/iva_inst/assert_2	Concurrent	SVA	on	0	0	-	0B	0B	0 ns	0	off	assert(@(posedge vif.clk) (vif.cou...	✓
/counter_top/iva_inst/assert_3	Concurrent	SVA	on	0	0	-	0B	0B	0 ns	0	off	assert(@(posedge vif.clk) (vif.cou...	✓

Coverage Report by instance with details

=====

=== Instance: /counter_top/intf

=== Design Unit: work.counter_if

=====

Toggle Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	----	-----
Toggles	30	30	0	100.00%

=====Toggle Details=====

Toggle Coverage for instance /counter_top/intf --

Node	1H->0L	0L->1H	"Coverage"
-----	-----	-----	-----
ce	1	1	100.00
clk	1	1	100.00
count_out[3-0]	1	1	100.00
data_load[3-0]	1	1	100.00
load_n	1	1	100.00
max_count	1	1	100.00
rst_n	1	1	100.00
up_down	1	1	100.00
zero	1	1	100.00

Total Node Count = 15

Toggled Node Count = 15

Untoggled Node Count = 0

Toggle Coverage = 100.00% (30 of 30 bins)

=====

=== Instance: /counter_top/dut/binding_inst

=== Design Unit: work.counter_sva

=====

Assertion Coverage:

Assertions	6	6	0	100.00%
------------	---	---	---	---------

```
=====
=== Instance: /counter_top/dut/binding_inst
=== Design Unit: work.counter_sva
=====
```

Assertion Coverage:

Assertions	6	6	0	100.00%
Name	File(Line)	Failure Count	Pass Count	
/counter_top/dut/binding_inst/a_load	counter_sva.sv(8)	0	1	
/counter_top/dut/binding_inst/a_increment	counter_sva.sv(31)	0	1	
/counter_top/dut/binding_inst/a_decrement	counter_sva.sv(40)	0	1	
/counter_top/dut/binding_inst/a_reset	counter_sva.sv(46)	0	1	
/counter_top/dut/binding_inst/a_max_count	counter_sva.sv(55)	0	1	
/counter_top/dut/binding_inst/a_zero	counter_sva.sv(63)	0	1	

Branch Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Branches	2	2	0	100.00%

=====Branch Details=====

Branch Coverage for instance /counter_top/dut/binding_inst

NOTE: The modification timestamp for source file 'counter_sva.sv' has been altered since compilation.

Line	Item	Count	Source
----	----	-----	-----
File counter_sva.sv			
-----IF Branch-----			
45		18	Count coming in to IF
45	1	2	(vif.count_out == 4'b1111) -> vif.max_count;
		16	All False Count

Branch totals: 2 hits of 2 branches = 100.00%

Statements	1	1	0	100.00%
------------	---	---	---	---------

=====Statement Details=====

Statement Coverage for instance /counter_top/dut/binding_inst --

NOTE: The modification timestamp for source file 'counter_sva.sv' has been altered since compilation.

Line	Item	Count	Source
----	----	-----	-----
File counter_sva.sv			
1			module counter_sva(counter_if.SVA vif);
2			
3			// i. When load is active, dout equals din
4			property p_load;
5			@(posedge vif.clk) disable iff (!vif.rst_n)
6			!vif.load_n -> (vif.count_out == vif.data_load);
7			endproperty
8			a_load: assert property (p_load);
9			c_load: cover property (p_load);
10			
11			// ii. When load inactive and enable off, dout doesn't change
12			property p_no_change;
13			@(posedge vif.clk) disable iff (!vif.rst_n)
14			(vif.load_n && !vif.ce && vif.rst_n) -> (vif.count_out == \$past(vif.count_out));
15			endproperty
16			
17			// iii. When load inactive, enable active, and up_down high, dout increments

```
=====
=== Instance: /counter_top/dut
=== Design Unit: work.counter
=====
```

Branch Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Branches	10	10	0	100.00%

```
=====Branch Details=====
```

Branch Coverage for instance /counter_top/dut

Line	Item	Count	Source
----	----	----	-----
File counter.sv			
-----IF Branch-----			
21		26	Count coming in to IF
21	1	2	if (!rst_n)
23	1	3	else if (!load_n)
25	1	12	else if (ce)
		9	All False Count

Branch totals: 4 hits of 4 branches = 100.00%

-----IF Branch-----			
26		12	Count coming in to IF
26	1	6	if (up_down)
28	1	6	else

Branch totals: 2 hits of 2 branches = 100.00%

-----IF Branch-----			
32		15	Count coming in to IF
32	1	3	assign max_count = (count_out == {WIDTH{1'b1}})? 1:0;
32	2	12	assign max_count = (count_out == {WIDTH{1'b1}})? 1:0;

Branch totals: 2 hits of 2 branches = 100.00%

Statement Coverage for instance /counter_top/dut --

Line	Item	Count	Source
-----	-----	-----	-----
File counter.sv			
8			module counter (clk ,rst_n, load_n, up_down, ce, data_load, count_out, max_count, zero);
9			parameter WIDTH = 4;
10			input clk;
11			input rst_n;
12			input load_n;
13			input up_down;
14			input ce;
15			input [WIDTH-1:0] data_load;
16			output reg [WIDTH-1:0] count_out;
17			output max_count;
18			output zero;
19			
20	1	26	always @(posedge clk) begin
21			if (!rst_n)
22	1	2	count_out <= 0;
23			else if (!load_n)
24	1	3	count_out <= data_load;
25			else if (ce)

asser_cvr_counter_top - Notepad

File Edit Format View Help

```
20          1          26  always @(posedge clk) begin
21
22          1          2      count_out <= 0;
23
24          1          3      count_out <= data_load;
25
26          else if (ce)
27          1          6      count_out <= count_out + 1;
28          else
29          1          6      count_out <= count_out - 1;
30
31          end
32          1          16  assign max_count = (count_out == {WIDTH{1'b1}})? 1:0;
33          1          16  assign zero = (count_out == 0)? 1:0;
```

Toggle Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Toggles	30	30	0	100.00%

=====Toggle Details=====

asser_cvr_counter_top - Notepad

File Edit Format View Help

```
32          1          16    assign max_count = (count_out == {WIDTH{1'b1}})? 1:0;

33          1          16    assign zero = (count_out == 0)? 1:0;
```

Toggle Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Toggles	30	30	0	100.00%

=====Toggle Details=====

Toggle Coverage for instance /counter_top/dut --

Node	1H->0L	0L->1H	"Coverage"
ce	1	1	100.00
clk	1	1	100.00
count_out[3-0]	1	1	100.00
data_load[0-3]	1	1	100.00
load_n	1	1	100.00
max_count	1	1	100.00
rst_n	1	1	100.00
up_down	1	1	100.00
zero	1	1	100.00

Total Node Count = 15
Toggled Node Count = 15
Untoggled Node Count = 0

Toggle Coverage = 100.00% (30 of 30 bins)

=====

```

1  ///////////////////////////////////////////////////////////////////
2  // Author: Kareem Waseem
3  // Course: Digital Verification using SV & UVM
4  //
5  // Description: Counter Design
6  //
7  ///////////////////////////////////////////////////////////////////
8  module counter (clk ,rst_n, load_n, up_down, ce, data_load, count_out, max_count, zero);
9  parameter WIDTH = 4;
10 input clk;
11 input rst_n;
12 input load_n;
13 input up_down;
14 input ce;
15 input [WIDTH-1:0] data_load;
16 output reg [WIDTH-1:0] count_out;
17 output max_count;
18 output zero;
19
20 always @(posedge clk) begin
21     if (!rst_n)
22         count_out <= 0;
23     else if (!load_n)
24         count_out <= data_load;
25     else if (ce)
26         if (up_down)
27             count_out <= count_out + 1;
28         else
29             count_out <= count_out - 1;
30 end
31
32 assign max_count = (count_out == {WIDTH{1'b1}})? 1:0;
33 assign zero = (count_out == 0)? 1:0;
34
35 endmodule

```

asser_cvr_counter_top - Notepad

File Edit Format View Help

Directive Coverage:

Directives	5	5	0	100.00%
------------	---	---	---	---------

DIRECTIVE COVERAGE:

Name	Design Unit	Design UnitType	Lang	File(Line)	Hits	Status
/counter_top/dut/binding_inst/c_load	counter_sva	Verilog	SVA	counter_sva.sv(9)	3	Covered
/counter_top/dut/binding_inst/c_increment	counter_sva	Verilog	SVA	counter_sva.sv(32)	6	Covered
/counter_top/dut/binding_inst/c_decrement	counter_sva	Verilog	SVA	counter_sva.sv(41)	6	Covered
/counter_top/dut/binding_inst/c_max_count	counter_sva	Verilog	SVA	counter_sva.sv(56)	14	Covered
/counter_top/dut/binding_inst/c_zero	counter_sva	Verilog	SVA	counter_sva.sv(64)	6	Covered

Statement Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
Statements	1	1	0	100.00%

Cover Directives

File Edit View Add Bookmarks Window Help

Cover Directives

Name	Language	Enabled	Log	Count	AtLeast	Limit	Weight	Cmplt %	Cmplt graph	Included	Memory	Peak Memory	Peak Memory Time	Cumulative Threads
/counter_top/dut/b... SVA	SVA	✓	Off	3	1	Unli...	1	100%		✓	0	0	0 ns	0
/counter_top/dut/b... SVA	SVA	✓	Off	6	1	Unli...	1	100%		✓	0	0	0 ns	0
/counter_top/dut/b... SVA	SVA	✓	Off	6	1	Unli...	1	100%		✓	0	0	0 ns	0
/counter_top/dut/b... SVA	SVA	✓	Off	14	1	Unli...	1	100%		✓	0	0	0 ns	0
/counter_top/dut/b... SVA	SVA	✓	Off	6	1	Unli...	1	100%		✓	0	0	0 ns	0

```
1 vlib work
2 vlog -sv *.sv +cover
3 vsim -voptargs=+acc counter_top -cover
4 add wave *
5 coverage save counter_top.ucdb -onexit
6 run -all
```

```

1  // counter_if.sv
2  interface counter_if(input clk);
3      logic rst_n;
4      logic load_n;
5      logic up_down;
6      logic ce;
7      logic [3:0] data_load;
8      logic [3:0] count_out;
9      logic max_count;
10     logic zero;
11
12     // Modport for DUT
13     modport DUT (
14         input clk, rst_n, load_n, up_down, ce, data_load,
15         output count_out, max_count, zero
16     );
17
18     // Modport for TB
19     modport TB (
20         output rst_n, load_n, up_down, ce, data_load,
21         input clk, count_out, max_count, zero
22     );
23
24     // Modport for SVA
25     modport SVA (
26         input clk, rst_n, load_n, up_down, ce, data_load, count_out, max_count, zero
27     );
28
29     // Modport for Monitor
30     modport MONITOR (
31         input clk, rst_n, load_n, up_down, ce, data_load, count_out, max_count, zero
32     );
33 endinterface

```

```

1  module counter_monitor(counter_if.MONITOR vif);
2
3      initial begin
4          $display("Time\t rst_n\t load_n\t ce\t up_down\t data_load\t count_out\t max_count\t zero");
5          $display("=====");
6      end
7
8      always @(posedge vif.clk) begin
9          $display("%0t\t %b\t %b\t %b\t %b\t %b\t %b\t %b\t %b",
10              $time, vif.rst_n, vif.load_n, vif.ce, vif.up_down,
11              vif.data_load, vif.count_out, vif.max_count, vif.zero);
12      end
13
14 endmodule

```

```

1  module counter_sva(counter_if.SVA vif);
2
3      // i. When load is active, dout equals din
4      property p_load;
5          @(posedge vif.clk) disable iff (!vif.rst_n)
6              !vif.load_n |-> (vif.count_out == vif.data_load);
7      endproperty
8      a_load: assert property (p_load);
9      c_load: cover property (p_load);
10
11     // ii. When load inactive and enable off, dout doesn't change
12     property p_no_change;
13         @(posedge vif.clk) disable iff (!vif.rst_n)
14             (vif.load_n && !vif.ce && vif.rst_n) |-> (vif.count_out == $past(vif.count_out));
15     endproperty
16
17     // iii. When load inactive, enable active, and up_down high, dout increments
18     property p_increment;
19         @(posedge vif.clk) disable iff (!vif.rst_n)
20             (vif.load_n && vif.ce && vif.up_down) |->
21                 (vif.count_out == $past(vif.count_out) + 1'b1);
22     endproperty
23     a_increment: assert property (p_increment);
24     c_increment: cover property (p_increment);
25
26     // iv. When load inactive, enable active, and up_down low, dout decrements
27     property p_decrement;
28         @(posedge vif.clk) disable iff (!vif.rst_n)
29             (vif.load_n && vif.ce && !vif.up_down) |->
30                 (vif.count_out == $past(vif.count_out) - 1'b1);
31     endproperty
32     a_decrement: assert property (p_decrement);
33     c_decrement: cover property (p_decrement);

```

```

29         (vif.load_n && vif.ce && !vif.up_down) |->
30         (vif.count_out == $past(vif.count_out) - 1'b1);
31     endproperty
32     a_decrement: assert property (p_decrement);
33     c_decrement: cover property (p_decrement);
34
35     // v. Async reset check using assert final
36     always_comb begin
37         if(!vif.rst_n) begin
38             a_reset: assert final(vif.count_out == 0);
39         end
40     end
41
42     // vi. max_count output is high when counter output is maximum
43     property p_max_count;
44         @(posedge vif.clk) disable iff (!vif.rst_n)
45         (vif.count_out == 4'b1111) |-> vif.max_count;
46     endproperty
47     a_max_count: assert property (p_max_count);
48     c_max_count: cover property (p_max_count);
49
50     // vii. zero output is high when counter output is zero
51     property p_zero;
52         @(posedge vif.clk) disable iff (!vif.rst_n)
53         (vif.count_out == 0) |-> vif.zero;
54     endproperty
55     a_zero: assert property (p_zero);
56     c_zero: cover property (p_zero);
57
58     endmodule
59

```

```
1  module counter_tb(counter_if.TB vif);
2
3      initial begin
4          // Initialize signals
5          vif.rst_n = 1'b1;
6          vif.load_n = 1'b1;
7          vif.up_down = 1'b1;
8          vif.ce = 1'b0;
9          vif.data_load = 4'b0;
10
11         // Apply reset
12         #15 vif.rst_n = 1'b0;
13         #20 vif.rst_n = 1'b1;
14
15         // Test load functionality
16         #10 vif.data_load = 4'b1010;
17         vif.load_n = 1'b0;
18         #10 vif.load_n = 1'b1;
19
20         // Test increment
21         #10 vif.ce = 1'b1;
22         vif.up_down = 1'b1;
23         #50 vif.ce = 1'b0;
24
25         // Test decrement
26         #10 vif.ce = 1'b1;
27         vif.up_down = 1'b0;
28         #50 vif.ce = 1'b0;
29
30         // Test boundary conditions
31         #10 vif.load_n = 1'b0;
32         vif.data_load = 4'b1111; // Max value
33         #10 vif.load_n = 1'b1;
34         #10 vif.ce = 1'b1;
35         vif.up_down = 1'b1;
36         #10 vif.ce = 1'b0;
```

```

24
25 // Test decrement
26 #10 vif.ce = 1'b1;
27 vif.up_down = 1'b0;
28 #50 vif.ce = 1'b0;
29
30 // Test boundary conditions
31 #10 vif.load_n = 1'b0;
32 vif.data_load = 4'b1111; // Max value
33 #10 vif.load_n = 1'b1;
34 #10 vif.ce = 1'b1;
35 vif.up_down = 1'b1;
36 #10 vif.ce = 1'b0;
37
38 #10 vif.load_n = 1'b0;
39 vif.data_load = 4'b0000; // Zero value
40 #10 vif.load_n = 1'b1;
41 #10 vif.ce = 1'b1;
42 vif.up_down = 1'b0;
43 #10 vif.ce = 1'b0;
44
45 // Finish simulation
46 □ #100 $finish;
47 end
48
49 endmodule

```

```

1  module counter_top;
2
3      bit clk;
4      always #5 clk = ~clk;
5
6      counter_if intf(clk);
7
8      counter dut (
9          .clk(intf.clk),
10         .rst_n(intf.rst_n),
11         .load_n(intf.load_n),
12         .up_down(intf.up_down),
13         .ce(intf.ce),
14         .data_load(intf.data_load),
15         .count_out(intf.count_out),
16         .max_count(intf.max_count),
17         .zero(intf.zero)
18     );
19
20     counter_tb tb(.vif(intf.TB));
21     counter_monitor mon(.vif(intf.MONITOR));
22
23     // Bind SVA to DUT
24     bind counter counter_sva binding_inst(.vif(intf.SVA));
25
26     initial begin
27         $dumpfile("counter.vcd");
28         $dumpvars(0, counter_top);
29         #500 $finish;
30     end
31
32 endmodule

```

```

# Time    rst_n    load_n    ce      up_down  data_load  count_out  max_count  zero
# =====
# 5       1       1       0       1          0000      xxxx      x          x
# 15      0       1       0       1          0000      xxxx      x          x
# 25      0       1       0       1          0000      0000      0          1
# 35      1       1       0       1          0000      0000      0          1
# 45      1       0       0       1          1010      0000      0          1
# 55      1       1       0       1          1010      1010      0          0
# 65      1       1       1       1          1010      1010      0          0
# 75      1       1       1       1          1010      1011      0          0
# 85      1       1       1       1          1010      1100      0          0
# 95      1       1       1       1          1010      1101      0          0
# 105     1       1       1       1          1010      1110      0          0
# 115     1       1       0       1          1010      1111      1          0
# 125     1       1       1       0          1010      1111      1          0
# 135     1       1       1       0          1010      1110      0          0
# 145     1       1       1       0          1010      1101      0          0
# 155     1       1       1       0          1010      1100      0          0
# 165     1       1       1       0          1010      1011      0          0
# 175     1       1       0       0          1010      1010      0          0
# 185     1       0       0       0          1111      1010      0          0
# 195     1       1       0       0          1111      1111      1          0
# 205     1       1       1       1          1111      1111      1          0
# 215     1       1       0       1          1111      0000      0          1
# 225     1       0       0       1          0000      0000      0          1
# 235     1       1       0       1          0000      0000      0          1
# 245     1       1       1       0          0000      0000      0          1
# 255     1       1       0       0          0000      1111      1          0
# 265     1       1       0       0          0000      1111      1          0
# 275     1       1       0       0          0000      1111      1          0
# 285     1       1       0       0          0000      1111      1          0
# 295     1       1       0       0          0000      1111      1          0
# 305     1       1       0       0          0000      1111      1          0
# 315     1       1       0       0          0000      1111      1          0
# 325     1       1       0       0          0000      1111      1          0
# 335     1       1       0       0          0000      1111      1          0
# 345     1       1       0       0          0000      1111      1          0
# ** Note: $finish      : counter_tb.sv(46)
#   Time: 355 ns  Iteration: 0  Instance: /counter_top/tb
.

```

Q4

```

1  module config_reg(
2      input  logic clk,
3      input  logic reset,
4      input  logic write,
5      input  logic [15:0] data_in,
6      input  logic [2:0] address,
7      output logic [15:0] data_out
8  );
9
10 `protected
11
12     MTI!#DU;wt7W\E}vp$.h,RJ@F"cok2*:fu| ]?KQ7"kDn2YBkJW1x7+D][a92\T\Xo+#4'mT1<VEU
13     Y>-jEo-mnC;onUwzW]kB*s~TB1VA61@Ba)+7?|,5XoUXY[@7?]AEK#5u2[,vCY?5WACBAKE-5Q<A
14     \1das*pl7+WpVrAz@eu$WA{ebXUv5TrV[v$A1+IB~*kJK~TXv^Vw@]DQ]uzjv(G,i{LY}DZ$4^O,
15     {cQ#=/_xJpe_kwG?wE4qLAR5TU$3sI(^E#*,iTK7'RVBKXe\YOiZwxm1mAjGo2]1_1uC2}##1#I
16     P#a7n^|e11LPZ*1[3rTr=iEkb@T>J-pmux,$Z.\vQ2[-1Jajw\sjirkh-'$[Ko7r*?\?+$WWAB_~,
17     ?e_<@'aB.a=Yi7aXRznB;\7ou~=!OV-p2^mDDu'sRiAn!m[J{Ho#-e1=Y7VDVT[G?v=$nL[:Kaw!
18     EZTH}xUz5KaliVG][KZ2evwm~|tZIk3xeGs2o[GG]p[BAW!w>eQaTz*R1#TxX{n>^}!EC$ekE#Z7
19     ,B5swuxN^H\2w>3Y=2E13}xW2xWQk5[#Qviz*w1@s+^UYy=-_nUw5s@-V2B-CjTp]Rw)e<UawXJ
20     3+H*vpW@U{x{$R@W^$>},E$v#BR{a+,uWuA}WxQ}X'JX[O5xkbl#B#/DXo[*wV,FqZXmAy=I-^n
21     Br*@5+[Jzoh,$11KG>^9J*e~Hr?'TCNH]Bw~1[u@e$u>Q23#BR'$Wxl_]IpI*{J5?OKDrHKA$GsG
22     a<j^D{*^Q^XzeaXL$Bsow<3*5lXBz*W*Z$CXMPK^zCvpDU5DT1aTJ=_Das=>o-TCs?aDE1}U5irC
23     <A,~vQi}QOuo;!VY@]C-+ROh'?Ku'pxvE!2(.Ss3JD'Q'eD?Esjp^[6]Z+u3n1}1urmi}@$eiDk
24     l2\+73l-ExuR,n<*I!oW^;1BE;*DBxTpVwXBhq][<)=KoTyU>2jsY]R!U[sD+O>_{1U^Ev!27T\Y
25     s}1q*$5T6kv!!hLEIwsX'CXe#GeQ*Y'[OvRkpr\A1oxGHpoW'lj]xRpjG!oTem?Q,mIp'7^11G]p
26     A*mTLj; \jrv[V*ps_T*jdrBX]tKE?X#n@<w[kp5XH]FNwoTBC]]Ud;5D[?][5D15]Tt_{ }o#-Q*
27     ^ep=(IH<oPD)QBjr<1p- '{sm5z@HBG,#rCiEQY}_1r[>;rr]u,wCKpGu)*w>lvvBr#j>s3Bz^I2x
28     <\jOW1@Ii!H^[:!R=[ ]7iu=Z[am[jT>B>IkYmQVyg0,VAQ\E\[Y2*RR=w)o_O$BwT$I3G_OI[HT'
29     3v@DZH1kp'@H0~EoI''Kj3Wo'3*={jE$m*bo_}xBVs]E[psIrjABY+s]usrsi!\L#_#J/Vm\HVCe
30     <o)K]@5,Yo!+Re77K>G-'^#si7;KBo[a@=i
31 `endprotected
32
33
34 endmodule // test
35

```

```

1  vlib work
2  vlog config_reg_buggy_questa.svp config_reg_tb.sv
3  vsim config_reg_tb
4  add wave *
5  run -all

```

```

1  module config_reg_tb;
2      logic clk, reset, write;
3      logic [15:0] data_in;
4      logic [2:0] address;
5      logic [15:0] data_out;
6
7      config_reg dut(.*);
8
9      // Clock generation
10     always #5 clk = ~clk;
11
12     // Enum for register addresses
13     typedef enum logic [2:0] {
14         ADC0_REG = 3'd0,
15         ADC1_REG = 3'd1,
16         TEMP_SENSOR0_REG = 3'd2,
17         TEMP_SENSOR1_REG = 3'd3,
18         ANALOG_TEST = 3'd4,
19         DIGITAL_TEST = 3'd5,
20         AMP_GAIN = 3'd6,
21         DIGITAL_CONFIG = 3'd7
22     } reg_address_t;
23
24     // Associative array for reset values
25     logic [15:0] reset_assoc[string];
26
27     // Test variables
28     reg_address_t addr_enum;
29     logic [15:0] expected_value;
30     int error_count = 0;
31
32     // Initialize reset values
33     initial begin
34         reset_assoc["adc0_reg"] = 16'hFFFF;

```

```

32 // Initialize reset values
33 initial begin
34     reset_assoc["adc0_reg"] = 16'hFFFF;
35     reset_assoc["adc1_reg"] = 16'h0;
36     reset_assoc["temp_sensor0_reg"] = 16'h0;
37     reset_assoc["temp_sensor1_reg"] = 16'h0;
38     reset_assoc["analog_test"] = 16'hABCD;
39     reset_assoc["digital_test"] = 16'h0;
40     reset_assoc["amp_gain"] = 16'h0;
41     reset_assoc["digital_config"] = 16'h1;
42 end
43
44 // Reset task
45 task apply_reset();
46     reset = 1'b1;
47     write = 1'b0;
48     data_in = 16'h0;
49     address = 3'd0;
50     @(negedge clk);
51     reset = 1'b0;
52     @(negedge clk);
53 endtask
54
55 // Check result task
56 task check_result(string reg_name, logic [15:0] expected, logic [15:0] actual);
57     if (expected != actual) begin
58         $display("ERROR: %s - Expected: %h, Actual: %h", reg_name, expected, actual);
59         error_count++;
60     end else begin
61         $display("PASS: %s - Value: %h", reg_name, actual);
62     end
63 endtask

```

```

64
65 // Write task
66 task write_register(reg_address_t addr, logic [15:0] data);
67     @(negedge clk);
68     address = addr;
69     data_in = data;
70     write = 1'b1;
71     @(negedge clk);
72     write = 1'b0;
73 endtask
74
75 // Read task
76 task read_register(reg_address_t addr, output logic [15:0] data);
77     @(negedge clk);
78     address = addr;
79     @(negedge clk);
80     data = data_out;
81 endtask
82
83 // Main test sequence
84 initial begin
85     // Initialize
86     clk = 0;
87     apply_reset();
88
89     // Test 1: Verify reset values
90     $display("=== Testing Reset Values ===");
91     for (addr_enum = addr_enum.first; addr_enum <= addr_enum.last; addr_enum = addr_enum.next) begin
92         logic [15:0] read_data;
93         read_register(addr_enum, read_data);
94
95         case (addr_enum)
96             ADC0_REG: check_result("adc0_reg", reset_assoc["adc0_reg"], read_data);
97             ADC1_REG: check_result("adc1_reg", reset_assoc["adc1_reg"], read_data);
98             TEMP_SENSOR0_REG: check_result("temp_sensor0_reg", reset_assoc["temp_sensor0_reg"], read_data);
99
100             case (addr_enum)
101                 ADC0_REG: check_result("adc0_reg", reset_assoc["adc0_reg"], read_data);
102                 ADC1_REG: check_result("adc1_reg", reset_assoc["adc1_reg"], read_data);
103                 TEMP_SENSOR0_REG: check_result("temp_sensor0_reg", reset_assoc["temp_sensor0_reg"], read_data);
104                 TEMP_SENSOR1_REG: check_result("temp_sensor1_reg", reset_assoc["temp_sensor1_reg"], read_data);
105                 ANALOG_TEST: check_result("analog_test", reset_assoc["analog_test"], read_data);
106                 DIGITAL_TEST: check_result("digital_test", reset_assoc["digital_test"], read_data);
107                 AMP_GAIN: check_result("amp_gain", reset_assoc["amp_gain"], read_data);
108                 DIGITAL_CONFIG: check_result("digital_config", reset_assoc["digital_config"], read_data);
109             endcase
110         end
111
112     // Test 2: Basic read/write test
113     $display("=== Testing Read/Write Functionality ===");
114     for (addr_enum = addr_enum.first; addr_enum <= addr_enum.last; addr_enum = addr_enum.next) begin
115         static logic [15:0] write_data = 16'hA5A5;
116         logic [15:0] read_data;
117
118         write_register(addr_enum, write_data);
119         read_register(addr_enum, read_data);
120
121         case (addr_enum)
122             ADC0_REG: check_result("adc0_reg write", write_data, read_data);
123             ADC1_REG: check_result("adc1_reg write", write_data, read_data);
124             TEMP_SENSOR0_REG: check_result("temp_sensor0_reg write", write_data, read_data);
125             TEMP_SENSOR1_REG: check_result("temp_sensor1_reg write", write_data, read_data);
126             ANALOG_TEST: check_result("analog_test write", write_data, read_data);
127             DIGITAL_TEST: check_result("digital_test write", write_data, read_data);
128             AMP_GAIN: check_result("amp_gain write", write_data, read_data);
129             DIGITAL_CONFIG: check_result("digital_config write", write_data, read_data);
130         endcase
131     end
132 end

```

```

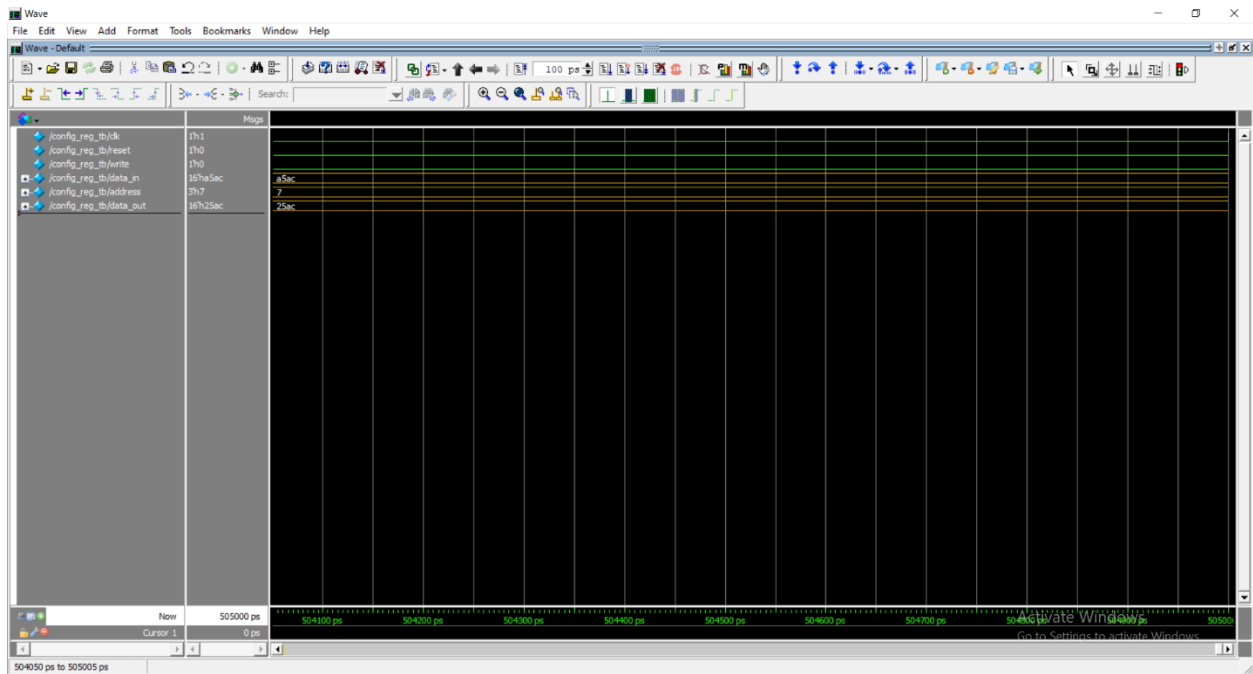
127
128 // Test 3: Bit-level testing
129 $display("=== Testing Bit-Level Writeability ===");
130 for (addr_enum = addr_enum.first; addr_enum <= addr_enum.last; addr_enum = addr_enum.next) begin
131     for (int bit_num = 0; bit_num < 16; bit_num++) begin
132         automatic logic [15:0] test_pattern = (1 << bit_num);
133         logic [15:0] read_data;
134
135         write_register(addr_enum, test_pattern);
136         read_register(addr_enum, read_data);
137
138         if (read_data != test_pattern) begin
139             $display("ERROR: Register %0d, Bit %0d - Written: %h, Read: %h",
140                 | | | | addr_enum, bit_num, test_pattern, read_data);
141             error_count++;
142         end
143     end
144 end
145
146 // Summary
147 $display("=== Test Summary ===");
148 $display("Total errors: %0d", error_count);
149 if (error_count == 0) begin
150     $display("ALL TESTS PASSED!");
151 end else begin
152     $display("SOME TESTS FAILED!");
153 end
154
155 $finish;
156 end
157

```

```

// Monitor
initial begin
    $timeformat(-9, 0, " ns", 10);
    $monitor("Time: %t | Address: %h | Write: %b | DataIn: %h | DataOut: %h",
        | | | | | $time, address, write, data_in, data_out);
end
endmodule

```



Verification plans

ALSU_1	When ALU is ADD or MUL, A and B should take all permutations of maxpos, maxneg, and zero	Randomized	Cross Coverage	Checker in testbench
ALSU_2	When ALU is ADD, c_in should take 0 and 1	Randomized	Coverpoint on c_in	Checker in testbench
ALSU_3	When ALU is shifting or rotating, direction must take 0 and 1	Randomized	Coverpoint on direction	Checker in testbench
ALSU_4	When ALU is shifting, shift_in must take 0 and 1	Randomized	Coverpoint on shift_in	Checker in testbench
ALSU_5	When ALU is OR or XOR and red_op_A=1, A must have walking one pattern (001,010,100) and B=0	Randomized	Cross Coverage	Checker in testbench
ALSU_6	When ALU is OR or XOR and red_op_B=1, B must have walking one pattern (001,010,100) and A=0	Randomized	Cross Coverage	Checker in testbench
ALSU_7	Invalid case: reduction active while opcode is not OR or XOR	Directed	Coverpoint + Illegal bin	Assertion

Label	Requirement	Assertion Type
A1	If a is high, b should be high after 2 clock cycles	Immediate
A2	If a and b are high, c should be high 1 to 3 cycles later	Temporal
A3	Sequence s11b: After 2 positive edges, b should be low	Sequence
A4.1	Decoder output Y: Only one bit high each clock cycle	Property
A4.2	Priority Encoder valid: If all inputs are low, valid must be low after 1 clock cycle	Property

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
COUNTER_1	When reset is asserted, output should go low immediately	Directed	-	SVA reset_assert
COUNTER_2	When load=1, output equals din	Randomized	-	SVA load_assert
COUNTER_3	When load=0, enable=0, output doesn't change	Randomized	-	SVA hold_assert
COUNTER_4	When load=0, enable=1, up_down=1, output increments by 1'b1	Randomized	-	SVA inc_assert
COUNTER_5	When load=0, enable=1, up_down=0, output decrements by 1'b1	Randomized	-	SVA dec_assert
COUNTER_6	max_count=1 when counter equals max value	Directed	Coverpoint	SVA max_count_assert
COUNTER_7	zero=1 when counter equals zero	Directed	Coverpoint	SVA zero_assert

Label	Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
CONF_REG_1	Verify reset values for all registers	Directed	Coverpoint	Checker in TB
CONF_REG_2	Write and read each register	Directed	Coverpoint	Checker in TB
CONF_REG_3	Detect bugs in each register	Directed	-	Manual debug